

Τεχνολογίες & Αλγόριθμοι
Αποκεντρωμένων Δεδομένων
Εξαμηνιαίο Project 2026



Δερεκενάρης Νικόλαος (1100529)

4ο έτος, up1100529@ac.upatras.gr

Κεφαλωνίτης Αργύριος (1100579)

4ο έτος, up1100579@ac.upatras.gr

Μητρογιάννης Ευάγγελος (1100626)

4ο έτος, up1100626@ac.upatras.gr

Νικολόπουλος Χρήστος Μάριος (100644)

4ο έτος, up100644@ac.upatras.gr

Τμήμα Μηχανικών Ηλεκτρονικών Υπολογιστών και Πληροφορικής

Πανεπιστήμιο Πατρών

Χειμερινό Εξάμηνο 2025-26

Contents

1	Εισαγωγή	4
1.1	Περιγραφή και Κίνητρο της Εργασίας	4
1.2	Στόχοι και Αντικείμενο	4
2	Frontend	5
2.1	Εισαγωγή	5
2.1.1	Σκοπός του Έργου	5
2.1.2	Ανάλυση Απαιτήσεων Frontend	6
2.2	Τεχνολογικό Υπόβαθρο	6
2.2.1	React.js και Vite	6
2.2.2	Βιβλιοθήκες και Εργαλεία	7
2.3	Αρχιτεκτονική Εφαρμογής	7
2.3.1	Δομή Συστήματος Αρχείων	7
2.3.2	Σημείο Εισόδου (Entry Point)	8
2.3.3	Κεντρική Δρομολόγηση και Ασφάλεια	8
2.4	Διεπαφή Χρήστη και Αναζήτηση	10
2.4.1	Σχεδιασμός UI και CSS Variables	10
2.4.2	Δυναμική Πλοήγηση (Navbar)	11
2.4.3	Λογική Φιλτραρίσματος (Home Page)	11
2.5	Ενσωμάτωση Machine Learning και Spark	13
2.5.1	Δομή και Λειτουργία της Σελίδας Λεπτομερειών	13
2.5.2	Προβολή Παρόμοιων Μαθημάτων (Similarity)	15
2.6	Σύστημα Συστάσεων και Analytics	15
2.6.1	Καταγραφή Ιστορικού	15
2.6.2	Οπτικοποίηση Δεδομένων (Analytics Dashboard)	17
2.7	Διαχείριση και Admin Dashboard	18
2.7.1	Triggering ETL και ML Jobs	19
2.8	Στρατηγική Διαχείρισης Κατάστασης (State Management)	20
2.9	Διαχείριση Σφαλμάτων και Ανθεκτικότητα	21
2.10	Συμπεράσματα	21

3	Backend	22
3.1	Αρχιτεκτονική και Δομή	22
3.2	Μοντέλα Δεδομένων (Data Models)	22
3.2.1	Course Schema	22
3.2.2	User Schema	24
3.2.3	Recommendation Schema	26
3.3	Authentication και Authorization	27
3.3.1	JWT-Based Authentication	27
3.3.2	Role-Based Authorization	29
3.4	CRUD Operations και API Endpoints	29
3.4.1	Ανάκτηση Μαθημάτων με Φίλτρα	29
3.4.2	Σύστημα Συστάσεων	31
3.4.3	Διαχείριση Αγαπημένων Μαθημάτων	32
3.4.4	Ιστορικό Προβολής	33
3.5	Analytics και Στατιστικά	34
3.6	Ενσωμάτωση με Spark Layer	35
4	Harvester (Data Connectors)	37
4.1	Αρχιτεκτονική Connectors	37
4.2	Connector για Coursera	38
4.2.1	Διαδικασία Ταυτοποίησης	38
4.2.2	Συλλογή Δεδομένων Μαθημάτων	39
4.2.3	Ομαλοποίηση Δεδομένων	39
4.3	Connector για Microsoft Learn	40
4.3.1	Συλλογή Δεδομένων	40
4.3.2	Ομαλοποίηση Ετερογενών Δεδομένων	41
4.4	Δυναμική Ανακάλυψη Connectors	42
4.5	Συγχρονισμός Δεδομένων	42
4.5.1	Συγχρονισμός Μεμονωμένης Πηγής	42
4.5.2	Ομαδικός Συγχρονισμός	44
5	Spark Layer	45
5.1	System Architecture	45

5.2	Μεθοδολογία & Αλγόριθμοι	46
5.2.1	NLP Pipeline	46
5.2.2	TF-IDF	47
5.2.3	Αναπαράσταση Δεδομένων: Document-Term Matrix	48
5.3	Στρατηγικές Παραγωγής Συστάσεων (Recommendation Scenarios)	48
5.3.1	Σενάριο 0: Exact Cosine Similarity (Ground Truth)	49
5.3.2	Σενάριο 1: Approximate k-NN (MinHash LSH)	49
5.3.3	Σενάριο 2: Thematic Recommender (LDA + LSH)	51
5.3.4	Σενάριο 3: Υβριδικό Μοντέλο Συστάσεων (Hybrid Recommender)	52
5.3.5	Τελική Επιλογή και Κατάταξη	52
5.4	Θεματική Συσταδοποίηση (Topic-based Clustering)	53
5.5	Προσωποποιημένες Προτάσεις Χρηστών με χρήση ALS	55
5.5.1	Γενική Περιγραφή ALS	55
5.5.2	Εφαρμογή και υλοποίηση στην πρόταση μαθημάτων	56
5.6	Τεχνικές Προκλήσεις και Επίλυση Προβλημάτων	56
5.6.1	Μετάβαση από Windows σε WSL	56
5.6.2	Διαχείριση Αναγνωριστικών MongoDB (<code>_id</code>)	57
5.6.3	Ασυμβατότητα Βιβλιοθηκών <code>pyspark.ml</code> και <code>pyspark.mllib</code>	57
5.7	Σχεδιαστικές Αποφάσεις και Μελλοντική Επεκτασιμότητα	58
5.8	Τελική Προσέγγιση: Triggered Batch Architecture (Admin-Driven)	59
6	Συμπεράσματα και Συνολική Αποτίμηση	60
6.1	Η Σύγκλιση των Τεχνολογιών	60
6.2	Τελικό Συμπέρασμα	60
7	Github Repository	61

1 Εισαγωγή

1.1 Περιγραφή και Κίνητρο της Εργασίας

Στην εποχή της ψηφιακής εκπαίδευσης, η πληθώρα των διαθέσιμων διαδικτυακών μαθημάτων έχει δημιουργήσει ένα τεράστιο τοπίο πληροφόρησης. Οι εκπαιδευόμενοι συχνά καλούνται να αναζητήσουν εκπαιδευτικό υλικό σε πολλαπλές, ετερογενείς πλατφόρμες (π.χ. Coursera, Udemy, EdX), γεγονός που καθιστά τη διαδικασία εύρεσης του κατάλληλου μαθήματος χρονοβόρα και δύσκολη.

Η παρούσα εργασία πραγματεύεται τη σχεδίαση και υλοποίηση του **Course Aggregator**, ενός «οριζόντιου αποθετηρίου» (horizontal repository). Το σύστημα αυτό δεν φιλοξενεί το ίδιο το εκπαιδευτικό περιεχόμενο (βίντεο, σημειώσεις), αλλά λειτουργεί ως ένας ενδιάμεσος κόμβος που συγκεντρώνει (harvests) μεταδεδομένα από πολλαπλές εξωτερικές πηγές. Στόχος είναι η παροχή μίας ενιαίας διεπαφής αναζήτησης και η προσφορά προστιθέμενης αξίας μέσω προηγμένων υπηρεσιών Μηχανικής Μάθησης (Machine Learning).

1.2 Στόχοι και Αντικείμενο

Ο βασικός σκοπός του έργου είναι η ανάπτυξη ενός ολοκληρωμένου πληροφοριακού συστήματος που συνδυάζει σύγχρονες τεχνολογίες διαδικτύου και επεξεργασίας μεγάλων δεδομένων. Οι επιμέρους στόχοι αναλύονται ως εξής:

1. Συγκέντρωση και Ομογενοποίηση Δεδομένων (Data Aggregation):

Η δημιουργία μηχανισμών (connectors) για την άντληση δεδομένων από ετερογενή εξωτερικά API και ο μετασχηματισμός τους σε ένα ενιαίο σχήμα βάσης δεδομένων.

2. Ενιαία Διεπαφή Χρήστη (Unified Frontend):

Η ανάπτυξη μιας σύγχρονης διαδικτυακής εφαρμογής (SPA) με χρήση της βιβλιοθήκης React. Η διεπαφή επιτρέπει στον χρήστη την αναζήτηση μαθημάτων με λέξεις-κλειδιά, την εφαρμογή σύνθετων φίλτρων (γλώσσα, επίπεδο, πάροχος) και την προβολή αναλυτικών πληροφοριών.

3. Ευφυής Επεξεργασία Δεδομένων (Big Data & ML):

Η αξιοποίηση του πλαισίου Apache Spark για την εκτέλεση εργασιών μεγάλης κλίμακας. Συγκεκριμένα, υλοποιούνται αλγόριθμοι για:

- **Συστάσεις Μαθημάτων (Recommendations):** Εύρεση παρόμοιων μαθημάτων βάσει περιεχομένου (Content-Based Filtering) με χρήση τεχνικών NLP (TF-IDF, Cosine Similarity).
- **Ομαδοποίηση (Clustering):** Αυτόματη κατηγοριοποίηση μαθημάτων σε θεματικές ενότητες με χρήση αλγορίθμων όπως ο LDA (Latent Dirichlet Allocation).
- **Προσωποποίηση (Personalization):** Παραγωγή προτάσεων βάσει του ιστορικού και των προτιμήσεων του χρήστη (Collaborative Filtering/ALS).

2 Frontend

Το Frontend αναπτύχθηκε με τη χρήση της βιβλιοθήκης **React.js** και του εργαλείου **Vite**, προσφέροντας μια σύγχρονη, γρήγορη και αποκριτική διεπαφή χρήστη (Responsive UI). Η εφαρμογή υλοποιεί σύνθετες λειτουργίες όπως δυναμική αναζήτηση και φιλτράρισμα σε πραγματικό χρόνο, σύστημα ταυτοποίησης χρηστών με JWT, προσωποποιημένο πίνακα ελέγχου (Dashboard) και οπτικοποίηση αποτελεσμάτων Μηχανικής Μάθησης (Machine Learning). Ιδιαίτερη έμφαση δόθηκε στη διαχωρισμένη αρχιτεκτονική (Decoupled Architecture), όπου το Frontend επικοινωνεί με το Backend μέσω REST API, εξασφαλίζοντας επεκτασιμότητα και συντηρησιμότητα.

2.1 Εισαγωγή

2.1.1 Σκοπός του Έργου

Στόχος της εργασίας είναι η υλοποίηση ενός οριζόντιου συστήματος αναζήτησης μαθημάτων, το οποίο λειτουργεί ως ενδιάμεσος κόμβος μεταξύ των παρόχων εκπαίδευσης (π.χ. Coursera, Udemy) και των τελικών χρηστών. Το σύστημα δεν αποθηκεύει το περιεχόμενο των βίντεο, αλλά τα μεταδεδωμένα τους, επιτρέποντας την ενιαία αναζήτηση και σύγκριση.

Ο ρόλος του Frontend είναι κρίσιμος, καθώς καλείται να παρουσιάσει με φιλικό τρόπο τα αποτελέσματα σύνθετων αλγορίθμων που εκτελούνται στο παρασκήνιο (Clustering, Recommendations) και να προσφέρει εργαλεία διαχείρισης για τους διαχειριστές του συστήματος.

2.1.2 Ανάλυση Απαιτήσεων Frontend

Βάσει των προδιαγραφών σχεδιασμού, το υποσύστημα διεπαφής χρήστη καλύπτει τις εξής λειτουργικές απαιτήσεις:

1. **Αναζήτηση και Φιλτράρισμα:** Δυνατότητα αναζήτησης με λέξεις-κλειδιά και εφαρμογή φίλτρων (γλώσσα, επίπεδο, πηγή, θεματική).
2. **Προβολή Λεπτομερειών:** Εμφάνιση αναλυτικών πληροφοριών μαθήματος και παραπομπή στην αρχική πηγή.
3. **Συστήματα Συστάσεων:** Οπτική απεικόνιση των αποτελεσμάτων του Spark ML ("Similar Courses") στη σελίδα λεπτομερειών.
4. **Διαχείριση Χρηστών:** Υλοποίηση μηχανισμού εγγραφής (Sign Up) και σύνδεσης (Login) με διαχείριση συνεδρίας (Session).
5. **Admin Dashboard:** Ειδική σελίδα για την εκκίνηση διαδικασιών συγχρονισμού (Harvesting) και εκπαίδευσης μοντέλων ML.
6. **Analytics:** Γραφική απεικόνιση στατιστικών στοιχείων χρήσης και κατανομής μαθημάτων.

2.2 Τεχνολογικό Υπόβαθρο

Η επιλογή των εργαλείων έγινε με γνώμονα την απόδοση (Performance), την εμπειρία προγραμματιστή (Developer Experience) και τη δυνατότητα κλιμάκωσης.

2.2.1 React.js και Vite

Για την ανάπτυξη του Frontend επιλέχθηκε η βιβλιοθήκη **React (έκδοση 18.3.1)**. Η React επιτρέπει τη δημιουργία επαναχρησιμοποιήσιμων UI components, διευκολύνοντας τη συντήρηση του κώδικα.

Ως εργαλείο build χρησιμοποιήθηκε το **Vite** προσφέροντας:

- **Άμεση εκκίνηση Server:** Χρησιμοποιεί native ES Modules στον browser κατά το development.

- **Hot Module Replacement (HMR):** Οι αλλαγές στον κώδικα αντικατοπτρίζονται άμεσα χωρίς πλήρη ανανέωση της σελίδας.
- **Βελτιστοποιημένο Build:** Χρησιμοποιεί το Rollup για την παραγωγή του τελικού κώδικα (production build).

2.2.2 Βιβλιοθήκες και Εργαλεία

Το οικοσύστημα της εφαρμογής περιλαμβάνει τις εξής κύριες βιβλιοθήκες:

- **React Router DOM:** Διαχείριση πλοήγησης (Routing) σε περιβάλλον Single Page Application (SPA).
- **Axios:** Διενέργεια HTTP κλήσεων προς το REST API με υποστήριξη Interceptors και Promises.
- **Docker:** Containerization της εφαρμογής για εύκολη εγκατάσταση και εκτέλεση σε οποιοδήποτε περιβάλλον.

2.3 Αρχιτεκτονική Εφαρμογής

2.3.1 Δομή Συστήματος Αρχείων

Η οργάνωση του κώδικα ακολουθεί μια αρθρωτή (modular) δομή. Διαχωρίζονται τα Pages components που διαχειρίζονται δεδομένα, από τα UI components που αφορούν μόνο την εμφάνιση.

```
frontend/
|-- public/                # Στατικά αρχεία (εικόνες, icons)
|-- src/
|   |-- assets/            # Πόροι εφαρμογής (logos, images)
|   |-- components/        # Επαναχρησιμοποιήσιμα δομικά στοιχεία
|   |   |-- auth/          # Login, SignUp forms
|   |   |   |-- AuthStyles.css
|   |   |   |-- Login.jsx
|   |   |   |-- SignUp.jsx
|   |   |-- shared/        # Κοινά στοιχεία (Navbar, Loaders)
|   |   |   |-- Navbar.css
|   |   |   |-- Navbar.jsx
|   |   |-- welcome_page/  # Hero section, Search bars
|   |   |   |-- Home.css
|   |   |   |-- Home.jsx
```



```
| | |-- Error.css
| | |-- Error.jsx          # Component σφάλματος
| | |-- pages/             # Κύριες οθόνες της εφαρμογής
| | |-- AdminDashboard.css
| | |-- AdminDashboard.jsx
| | |-- Analytics.css
| | |-- Analytics.jsx
| | |-- CourseDetails.css
| | |-- CourseDetails.jsx
| | |-- Favorites.jsx
| | |-- History.jsx
| |-- App.jsx              # Κεντρικό routing & Layout
| |-- index.css            # Global styles & CSS Variables
| |-- main.jsx             # Entry point (React DOM)
|-- .dockerignore
|-- Dockerfile
|-- index.html
|-- package.json
|-- vite.config.js
```

2.3.2 Σημείο Εισόδου (Entry Point)

Το αρχείο `main.jsx` αποτελεί το σημείο εισόδου της εφαρμογής, όπου το React "κουμπώνει" στο DOM του browser. Χρησιμοποιείται το νέο API `createRoot` της έκδοσης 18, ενώ η εφαρμογή περικλείεται σε `React.StrictMode` για τον εντοπισμό πιθανών προβλημάτων κατά την ανάπτυξη.

```
----- src/main.jsx -----
1  import React from 'react'
2  import ReactDOM from 'react-dom/client'
3  import App from './App.jsx'
4  import './index.css'
5
6  ReactDOM.createRoot(document.getElementById('root')).render(
7    <React.StrictMode>
8      <App />
9    </React.StrictMode>
10 )
```

2.3.3 Κεντρική Δρομολόγηση και Ασφάλεια

Η διαχείριση της πλοήγησης γίνεται στο αρχείο `App.jsx`. Εδώ ορίζονται οι διαδρομές και εφαρμόζεται η λογική προστασίας μέσω του `Component ProtectedRoute`.

Το `ProtectedRoute` λειτουργεί ως `Wrapper`. Ελέγχει την ύπαρξη του JWT token στο `LocalStorage` και το ρόλο του χρήστη (User vs Admin). Αν ο χρήστης δεν έχει δικαίωμα πρόσβασης, ανακατευθύνεται αυτόματα.

```
src/App.jsx
1 import { BrowserRouter as Router, Routes, Route, Navigate } from
  ↳ 'react-router-dom';
2
3 // Component προστασίας διαδρομών
4 const ProtectedRoute = ({ children, isAdminRequired }) => {
5   // Ανάκτηση δεδομένων χρήστη από Local Storage
6   const userString = localStorage.getItem('user');
7
8   // Έλεγχος Αuthεντικοποίησης: Αν δεν υπάρχει χρήστης, ανακατεύθυνση στο Login
9   if (!userString) {
10     console.log("Access Denied: No user found");
11     return <Navigate to="/login" replace />;
12   }
13
14   const user = JSON.parse(userString);
15
16   // Έλεγχος Εξουσιοδότησης: Αν απαιτείται ρόλος Admin και ο χρήστης δεν είναι
17   if (isAdminRequired && user.role?.toLowerCase() !== 'admin') {
18     console.log("Access Denied: Admin role required");
19     return <Navigate to="/" replace />;
20   }
21
22   // Αν όλα είναι εντάξει, επιστροφή του παιδιού-component
23   return children;
24 };
25
26 function App() {
27   return (
28     <Router>
29     <div className="App">
30       <Navbar />
31       <Routes>
32         {/* Public Routes - Προσβάσιμες από όλους */}
33         <Route path="/" element={<Home />} />
34         <Route path="/login" element={<Login />} />
35         <Route path="/signup" element={<SignUp />} />
36         <Route path="/courses/:id" element={<CourseDetails />} />
37
38         {/* Protected Admin Route - Προσβάσιμη μόνο από Admin */}
39         <Route
40           path="/admin-dashboard"
41           element={
42             <ProtectedRoute isAdminRequired={true}>
43               <AdminDashboard />
```

```
44         </ProtectedRoute>
45     }
46     />
47
48     { /* Catch-all για 404 - Χειρισμός άγνωστων URL */ }
49     <Route path="*" element={<Error />} />
50 </Routes>
51 </div>
52 </Router>
53 );
54 }
```

2.4 Διεπαφή Χρήστη και Αναζήτηση

2.4.1 Σχεδιασμός UI και CSS Variables

Ιδιαίτερη έμφαση δόθηκε στη χρηστικότητα και την άμεση ανατροφοδότηση του συστήματος προς τον χρήστη:

1. **Responsive Design:** Η εφαρμογή είναι πλήρως προσαρμόσιμη σε διαφορετικά μεγέθη οθόνης (κινητά, tablets, desktops). Μέσω CSS Media Queries και ευέλικτων layouts (Flexbox/Grid), τα στοιχεία αναδιατάσσονται ώστε να παραμένουν ευανάγνωστα. Για παράδειγμα, το πλέγμα των καρτών μαθημάτων μετατρέπεται από 4 στήλες σε Desktop σε 1 στήλη σε κινητά τηλέφωνα.
2. **Visual Feedback:** Ο χρήστης λαμβάνει οπτική επιβεβαίωση για κάθε ενέργεια:
 - **Loading States:** Κατά τη διάρκεια ασύγχρονων κλήσεων (π.χ. φόρτωση μαθημάτων ή εκτέλεση Spark Job), εμφανίζονται ενδείξεις φόρτωσης (spinners) ή σκελετοί (skeletons) για να μειωθεί η αντιληπτή καθυστέρηση.
 - **Interactive Elements:** Τα κουμπιά και οι κάρτες έχουν hover effects για να υποδεικνύουν τη διαδραστικότητά τους.
 - **Empty States:** Σε περίπτωση που η αναζήτηση δεν επιστρέφει αποτελέσματα, εμφανίζεται φιλικό μήνυμα καθοδήγησης αντί για κενή οθόνη.

Για να εξασφαλιστεί η συνοχή του σχεδιασμού (Design Consistency), δημιουργήθηκε ένα σύστημα μεταβλητών CSS (Design Tokens) στο `index.css`. Αυτό επιτρέπει την εύκολη αλλαγή θεμάτων και τη διατήρηση ενιαίας παλέτας χρωμάτων.

2.4.2 Δυναμική Πλοήγηση (Navbar)

Το Component Navbar προσαρμόζει το περιεχόμενό του δυναμικά, ελέγχοντας αν ο χρήστης είναι συνδεδεμένος και αν έχει ρόλο διαχειριστή. Αυτό επιτυγχάνεται διαβάζοντας το `localStorage` και ενημερώνοντας το `state` κατά την πλοήγηση. Έτσι, το UI αλλάζει σε πραγματικό χρόνο χωρίς να χρειάζεται ανανέωση της σελίδας.

```
src/components/shared/Navbar.jsx
1  const Navbar = () => {
2    // State initialization logic omitted for brevity...
3
4    return (
5      <nav className="dropbox-navbar">
6        <div className="nav-right">
7          {/* Έλεγχος κατάστασης σύνδεσης (Authentication Check) */}
8          {isAuthenticated ? (
9            <>
10              {/* Conditional Rendering για Admin Link */}
11              {isAdmin && (
12                <Link to="/admin-dashboard" className="admin-link">
13                  Admin Panel
14                </Link>
15              )}
16              <Link to="/analytics">Analytics</Link>
17              <button onClick={handleLogout} className="btn-logout">
18                Αποσύνδεση
19              </button>
20            </>
21          ) : (
22            <>
23              <Link to="/login">Σύνδεση</Link>
24              <Link to="/signup" className="btn-signup">Εγγραφή</Link>
25            </>
26          )}
27        </div>
28      </nav>
29    );
30  };
```

2.4.3 Λογική Φιλτραρίσματος (Home Page)

Η σελίδα Home αποτελεί το κεντρικό σημείο αλληλεπίδρασης. Η κατάσταση (State) της σελίδας περιλαμβάνει τα πρωτογενή δεδομένα (`allCourses`), τα φιλτραρισμένα δεδομένα (`courses`) και τις επιλογές του χρήστη.

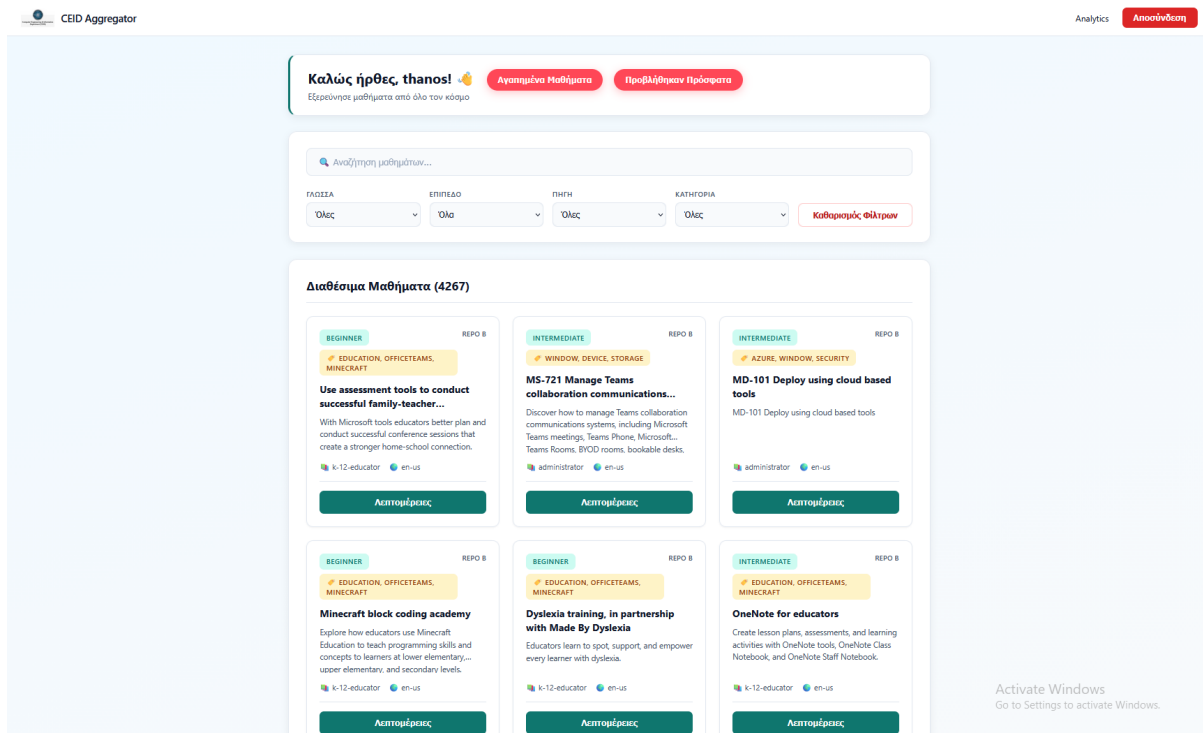


Figure 1: Η Αρχική Σελίδα της εφαρμογής με ενεργά φίλτρα και αποτελέσματα αναζήτησης.

Για τη βελτιστοποίηση της απόδοσης και την αποφυγή περιττών υπολογισμών σε κάθε render, οι λίστες των μοναδικών φίλτρων (γλώσσες, επίπεδα, πάροχοι) υπολογίζονται μέσω του hook `useMemo`. Αυτό σημαίνει ότι οι λίστες επαναυπολογίζονται μόνο όταν αλλάξουν τα δεδομένα των μαθημάτων και όχι σε κάθε ανανέωση της οθόνης.

```

src/pages/Home.jsx (Optimization)
1  // Υπολογισμός μοναδικών γλωσσών μόνο όταν αλλάξει το allCourses
2  const uniqueLanguages = useMemo(() =>
3    [...new Set(allCourses.map(c => c.language).filter(Boolean))],
4    [allCourses]);
5
6  const uniqueLevels = useMemo(() =>
7    [...new Set(allCourses.map(c => c.level).filter(Boolean))],
8    [allCourses]);
9
10 const uniqueCategories = useMemo(() =>
11    [...new Set(allCourses.map(c => c.category).filter(Boolean))],
12    [allCourses]);

```

Για την εφαρμογή των φίλτρων κατά την πληκτρολόγηση ή την επιλογή από τα dropdowns, χρησιμοποιείται το hook `useEffect`. Κάθε αλλαγή στα κριτήρια αναζήτησης (`searchTerm`) ή στα φίλτρα (`filters`) πυροδοτεί εκ νέου τον υπολογισμό της λίστας προς εμφάνιση (`result`).

```
src/pages/Home.jsx
1  useEffect(() => {
2    if (allCourses.length === 0) return;
3
4    let result = [...allCourses];
5
6    // 1. Φιλτράρισμα βάσει κειμένου (Search Bar) - Case Insensitive
7    if (searchTerm) {
8      const term = searchTerm.toLowerCase();
9      result = result.filter(course =>
10        (course.title || "").toLowerCase().includes(term) ||
11        (course.description || "").toLowerCase().includes(term)
12      );
13    }
14
15    // 2. Εφαρμογή δυναμικών φίλτρων (Dropdowns)
16    if (filters.language) {
17      result = result.filter(c =>
18        c.language && c.language.toLowerCase() ===
19        filters.language.toLowerCase()
20      );
21    }
22    if (filters.level) {
23      result = result.filter(c => c.level === filters.level);
24    }
25    if (filters.category) {
26      result = result.filter(c => c.category === filters.category);
27    }
28
29    // Ενημέρωση state με τα αποτελέσματα και επαναφορά σελίδας στο 1
30    setCourses(result);
31    setCurrentPage(1);
32  }, [searchTerm, filters, allCourses]); // Dependencies: Τρέχει όταν αλλάξει
33  ↳ κάποιο από αυτά
```

2.5 Ενσωμάτωση Machine Learning και Spark

Ένα από τα σημαντικότερα χαρακτηριστικά του συστήματος είναι η αξιοποίηση των αποτελεσμάτων που παράγονται από το Apache Spark. Το Frontend λαμβάνει αυτά τα δεδομένα μέσω του API και τα παρουσιάζει με φιλικό τρόπο.

2.5.1 Δομή και Λειτουργία της Σελίδας Λεπτομερειών

Η σελίδα `CourseDetails` αποτελεί τον πυρήνα της πληροφόρησης για τον τελικό χρήστη και το σημείο σύγκλισης των δεδομένων. Όταν επιλεγεί ένα μάθημα, η εφαρμογή μεταβαίνει

στη δυναμική διαδρομή /courses/:id, η οποία εξυπηρετεί τρεις παράλληλους στόχους:

1. **Προβολή Μεταδεδομένων:** Αντλεί και εμφανίζει τα πλήρη στοιχεία του μαθήματος (Τίτλος, Εκτενής Περιγραφή, Πάροχος, Γλώσσα, Link Εγγραφής).
2. **Καταγραφή Αλληλεπίδρασης (History Tracking):** Κατά τη φόρτωση (mount) του component, αποστέλλεται αίτημα στο Backend για να καταγραφεί η επίσκεψη στο ιστορικό του χρήστη. Αυτό το "σιωπηλό" δεδομένο τροφοδοτεί αργότερα τον αλγόριθμο ALS για προσωποποιημένες προτάσεις.
3. **Ανάδειξη Συσχετίσεων (Spark ML):** Καλεί εξειδικευμένα endpoints για να ανακτήσει και να προβάλει τα αποτελέσματα των αλγορίθμων ομοιότητας που έχουν παραχθεί από το Spark.

Αρχική / Use assessment tools to conduct successf...

BEGINNER REPO B EN-US EDUCATION, OFFICETEAMS, MINECRAFT

Use assessment tools to conduct successful family-teacher conferences

With Microsoft tools educators better plan and conduct successful conference sessions that create a stronger home-school connection.

Περιγραφή Μαθήματος

With Microsoft tools educators better plan and conduct successful conference sessions that create a stronger home-school connection.

Λέξεις-Κλειδιά

office-teams office-onenote m365-education

Λεπτομέρειες

Κατηγορία: k-12-educator

Τελευταία Ενημέρωση: 13/2/2026

Εγγραφή στο Μάθημα Προβολή στην Πηγή

Παρόμοια Μαθήματα

LEVEL	TAUTISIA (%)	COURSE TITLE	REPO
BEGINNER	70%	Countdown to successful family-teacher conferences	Repo B
BEGINNER	70%	Countdown to successful family-teacher conferences	Repo B
BEGINNER	30%	Microsoft educator academy	Repo B
BEGINNER	28%	Enhance school communication and collaboration in Microsoft Teams	Repo B
BEGINNER	28%	Support social and emotional learning with Microsoft tools	Repo B

Figure 2: Σελίδα λεπτομερειών μαθήματος με την ενότητα "Similar Courses" και τα ποσοστά ομοιότητας.

2.5.2 Προβολή Παρόμοιων Μαθημάτων (Similarity)

Στο πλαίσιο της ανάδειξης συσχετίσεων, το σύστημα καλεί το endpoint `/courses/{id}/similar`. Τα δεδομένα αυτά προέρχονται από αλγορίθμους όπως TF-IDF και Cosine Similarity που εκτελέστηκαν στο Spark Layer. Το Frontend όχι απλώς παραθέτει τα μαθήματα, αλλά εμφανίζει και το ποσοστό ταύτισης (Match Score), δίνοντας στον χρήστη μια ποσοτική ένδειξη για τη συνάφεια των προτάσεων.

```

src/pages/CourseDetails.jsx
1 {similarCourses.map((similar) => {
2   // Μετατροπή του cosine score (0-1) σε ποσοστό για εμφάνιση
3   const matchPercentage = similar.score ? Math.round((similar.score) * 100)
4     : null;
5
6   return (
7     // Κάρτα παρόμοιου μαθήματος με onClick handler για πλοήγηση
8     <div
9       key={similar._id}
10      className="similar-course-card"
11      onClick={() => navigate(`/courses/${similar._id}`)}
12    >
13      <div className="similar-course-header">
14        <span className="similar-level">{similar.level}</span>
15        {/* Conditional Rendering: Εμφάνιση Badge μόνο αν υπάρχει score */}
16        {matchPercentage && (
17          <span className="match-score">{matchPercentage}% Ταύτιση</span>
18        )}
19      </div>
20      <h4 className="similar-title">{similar.title}</h4>
21      <span className="similar-source">{similar.source}</span>
22    </div>
23  );
24 }}

```

2.6 Σύστημα Συστάσεων και Analytics

2.6.1 Καταγραφή Ιστορικού

Για να λειτουργήσουν τα συστήματα συστάσεων (όπως το Collaborative Filtering), το Frontend καταγράφει τη συμπεριφορά του χρήστη. Κάθε φορά που ανοίγει μια σελίδα μαθήματος, αποστέλλεται ένα "χτύπημα" στο Backend. Χρησιμοποιείται `useRef` για να διασφαλιστεί ότι το αίτημα στέλνεται μόνο μία φορά κατά το mounting του component (ιδιαίτερα χρήσιμο σε React StrictMode, όπου τα effects μπορεί να τρέξουν δύο φορές).

```
src/pages/CourseDetails.jsx
1  const hasTracked = useRef(false); // Ref για έλεγχο εκτέλεσης
2
3  useEffect(() => {
4    if (hasTracked.current) return; // Αποφυγή διπλής καταγραφής
5    hasTracked.current = true;
6
7    const trackVisit = async () => {
8      const token = localStorage.getItem('token');
9      if (!token) return; // Μόνο αν υπάρχει συνδεδεμένος χρήστης
10
11     try {
12       // POST Request για ενημέρωση του recentlyViewed
13       await axios.post(
14         `http://localhost:3000/courses/${id}/recentlyViewed`,
15         {},
16         { headers: { Authorization: `Bearer ${token}` } }
17       );
18     } catch (err) {
19       console.error("Error updating history", err);
20     }
21   };
22
23   trackVisit();
24 }, [id]); // Τρέχει κάθε φορά που αλλάζει το ID του μαθήματος
```

2.6.2 Οπτικοποίηση Δεδομένων (Analytics Dashboard)

Η σελίδα Analytics παρέχει μια εποπτική εικόνα της βάσης δεδομένων. Το Frontend αναχτά συγκεντρωτικά στατιστικά από το Backend και τα οπτικοποιεί χρησιμοποιώντας δυναμικές μπάρες (bar charts).

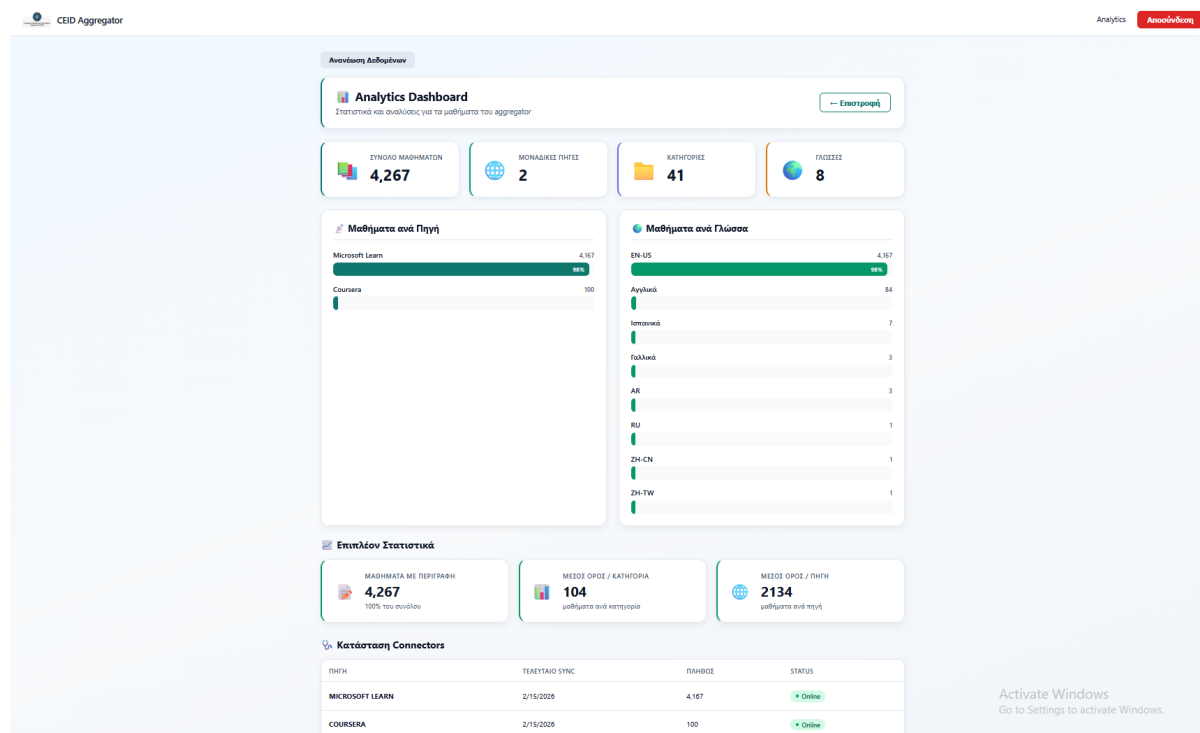


Figure 3: Οπτικοποίηση στατιστικών στοιχείων στη σελίδα Analytics.

Η ανάκτηση των δεδομένων γίνεται με ασφαλή τρόπο, ελέγχοντας την εγκυρότητα του token και διαχειρίζοντας τυχόν σφάλματα αυθεντικοποίησης (π.χ. λήξη token) με ανακατεύθυνση στο Login.

```

src/pages/Analytics.jsx
1  const fetchAnalytics = async () => {
2    try {
3      setLoading(true);
4      const token = localStorage.getItem('token');
5
6      // Κλήση στο προστατευμένο endpoint στατιστικών
7      const response = await
8        ↪ axios.get('http://localhost:3000/analytics/stats', {
9          headers: { Authorization: `Bearer ${token}` },
10         withCredentials: true
11       });

```

```

12     setStats(response.data);
13   } catch (error) {
14     // Διαχείριση σφάλματος 401 (Unauthorized)
15     if (error.response?.status === 401) {
16       navigate('/login'); // Redirect σε περίπτωση λήξης session
17     }
18   } finally {
19     setLoading(false);
20   }
21 };

```

Η οπτικοποίηση υλοποιείται με CSS-based charts για μέγιστη απόδοση, χωρίς τη χρήση βαριών βιβλιοθηκών γραφικών. Το πλάτος κάθε μπάρας υπολογίζεται δυναμικά βάσει του ποσοστού συμμετοχής της κατηγορίας.

```

_____ src/pages/Analytics.jsx (Render) _____
1  {stats.bySource.map((item, index) => (
2    <div key={index} className="chart-bar-item">
3      <div className="chart-bar-label">
4        <span className="label-name">{item.name}</span>
5        <span className="label-count">{item.count}</span>
6      </div>
7      <div className="chart-bar-container">
8        {/* Δυναμικός υπολογισμός πλάτους μπάρας βάσει του percentage */}
9        <div className="chart-bar" style={{ width: `${item.percentage}%` }}>
10         <span className="bar-percentage">{item.percentage}%</span>
11       </div>
12     </div>
13   </div>
14 ) ) }

```

2.7 Διαχείριση και Admin Dashboard

Το Admin Dashboard είναι το κέντρο ελέγχου της πλατφόρμας, προσβάσιμο μόνο από χρήστες με ρόλο admin. Παρέχει δυνατότητες ελέγχου των ETL διαδικασιών και των ML Pipelines.

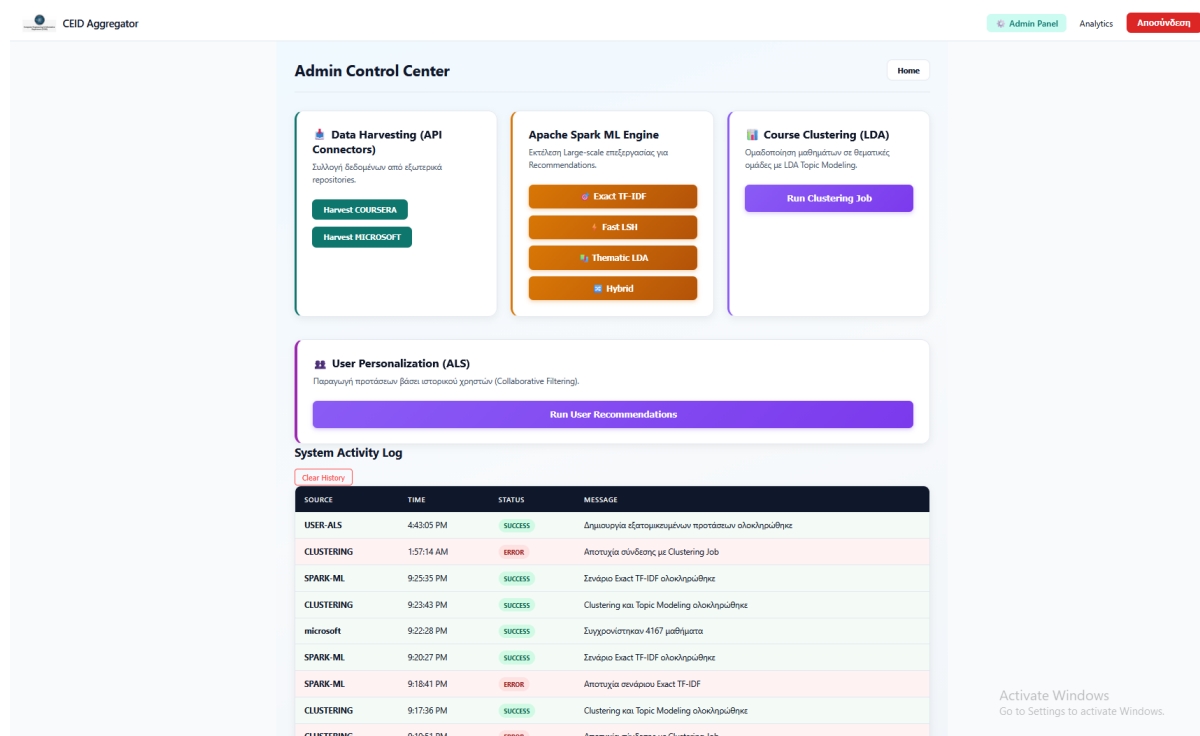


Figure 4: Πίνακας ελέγχου διαχειριστή με εργαλεία συγχρονισμού και εκτέλεσης ML σεναρίων.

2.7.1 Triggering ETL και ML Jobs

Το Frontend παρέχει κουμπιά ενεργειών που καλούν ασύγχρονα endpoints του Backend. Αυτά με τη σειρά τους εκκινούν Spark Jobs. Το σύστημα διατηρεί ένα τοπικό Log δραστηριότητας (Activity Log) στο localStorage για να ενημερώνει τον διαχειριστή για την έκβαση των εργασιών χωρίς να χρειάζεται να ελέγχει τα logs του server.

```

src/pages/AdminDashboard.jsx
1  const handleSync = async (source) => {
2    // Επιβεβαίωση ενέργειας από τον χρήστη
3    if (!window.confirm(`Έναρξη ETL process για: ${source.toUpperCase()}`))
4      return;
5
6    setIsSyncing(true);
7    const token = localStorage.getItem('token');
8
9    try {
10     // Κλήση στο Connector endpoint για έναρξη συγχρονισμού
11     const response = await
12       axios.get(`http://localhost:3000/courses/sync/${source}`, {
13         headers: { Authorization: `Bearer ${token}` }
14       });
15   }

```

```
13      // Καταγραφή επιτυχίας στο Activity Log
14      addLog(source, 'Success', `Συγχρονίστηκαν ${response.data.count}
      ↳ μαθήματα`);
15    } catch (error) {
16      // Καταγραφή αποτυχίας
17      addLog(source, 'Error', error.response?.data?.message || 'Αποτυχία
      ↳ connector');
18    } finally {
19      setIsSyncing(false);
20    }
21  };
22
23  const handleSparkJob = async (option, optionName) => {
24    // Option 0: TF-IDF, Option 1: LSH, Option 2: LDA, Option 3: Hybrid
25    setIsSparkRunning(true);
26    try {
27      // POST Request για εκκίνηση του Spark Job
28      await axios.post(`http://localhost:3000/analytics/run-spark/${option}`,
      ↳ {}, config);
29      addLog('SPARK-ML', 'Success', `Σενάριο ${optionName} ολοκληρώθηκε`);
30    } catch (error) {
31      addLog('SPARK-ML', 'Error', `Αποτυχία σεναρίου ${optionName}`);
32    }
33  };
```

2.8 Στρατηγική Διαχείρισης Κατάστασης (State Management)

Για τη διαχείριση της κατάστασης της εφαρμογής (Application State), επιλέχθηκε μια αποκεντρωμένη προσέγγιση αντί της χρήσης μιας καθολικής βιβλιοθήκης όπως το Redux. Η απόφαση αυτή βασίστηκε στην πολυπλοκότητα της εφαρμογής και στην ανάγκη για γρήγορη ανάπτυξη. Συγκεκριμένα:

- **Local State:** Χρησιμοποιείται το `useState` για δεδομένα που αφορούν αποκλειστικά ένα Component (π.χ. τιμή πεδίου αναζήτησης, άνοιγμα/κλείσιμο modal).
- **Persisted State:** Για την αυθεντικοποίηση (JWT token, user info) και τις προτιμήσεις του διαχειριστή (Activity Logs), χρησιμοποιείται το `localStorage`. Αυτό εξασφαλίζει ότι η κατάσταση διατηρείται ακόμα και μετά από ανανέωση της σελίδας (page refresh).
- **Server State:** Τα δεδομένα των μαθημάτων και των στατιστικών θεωρούνται "Server State" και ανανεώνονται μέσω του `useEffect` κατά την πλοήγηση, εξασφαλίζοντας

ότι ο χρήστης βλέπει πάντα φρέσκα δεδομένα χωρίς την ανάγκη περίπλοκου caching στον client.

2.9 Διαχείριση Σφαλμάτων και Ανθεκτικότητα

Το Frontend έχει σχεδιαστεί ώστε να διαχειρίζεται αποτυχίες του συστήματος χωρίς να "καταρρέει" (crash) η εφαρμογή:

- **API Failures:** Σε περίπτωση που το Backend δεν ανταποκρίνεται (π.χ. Network Error 500), η εφαρμογή εμφανίζει διακριτικά μηνύματα σφάλματος (Toast Notifications ή Error Banners) ενημερώνοντας τον χρήστη να δοκιμάσει αργότερα.
- **Broken Links/Routes:** Υλοποιήθηκε μια ειδική σελίδα "404 Not Found" (Catch-all route), ώστε αν ο χρήστης πληκτρολογήσει λάθος URL, να μπορεί να επιστρέψει εύκολα στην αρχική σελίδα.
- **Session Expiration:** Οι HTTP Interceptors ελέγχουν τις απαντήσεις του server. Αν ανιχνευθεί σφάλμα 401 (Unauthorized), το σύστημα αντιλαμβάνεται ότι το token έληξε, αποσυνδέει αυτόματα τον χρήστη και τον ανακατευθύνει στη σελίδα εισόδου.

2.10 Συμπεράσματα

Η υλοποίηση του Frontend του Aggregator πέτυχε τους στόχους που τέθηκαν στην αρχή του έργου.

- **Λειτουργικότητα:** Καλύφθηκαν πλήρως οι απαιτήσεις αναζήτησης, φιλτραρίσματος και προβολής μαθημάτων από ετερογενείς πηγές.
- **Διαλειτουργικότητα:** Η επικοινωνία με το Backend και η κατανάλωση των αποτελεσμάτων του Spark ML έγινε απρόσκοπτα, προσφέροντας προστιθέμενη αξία μέσω των συστάσεων.
- **Εμπειρία Χρήστη (UX):** Η χρήση της React και του μοντέρνου CSS Design System εξασφάλισε μια ευχάριστη και γρήγορη εμπειρία πλοήγησης.
- **Επεκτασιμότητα:** Η καθαρή αρχιτεκτονική (Component-based) και η χρήση Docker καθιστούν το σύστημα εύκολα συντηρήσιμο και επεκτάσιμο για μελλοντικές προσθήκες (π.χ. νέους παρόχους μαθημάτων).

3 Backend

Αναπτύχθηκε με χρήση του Node.js, και ακολουθεί την αρχιτεκτονική **MVC (Model-View-Controller)**, διαχωρίζοντας τη λογική σε ξεχωριστά επίπεδα για καλύτερη συντηρησιμότητα και επεκτασιμότητα.

3.1 Αρχιτεκτονική και Δομή

Η εφαρμογή ακολουθεί την αρχιτεκτονική MVC, η οποία διαχωρίζει τη λογική σε τέσσερα βασικά επίπεδα:

- **Models (Μοντέλα):** Ορίζουν τη δομή των δεδομένων μέσω Mongoose schemas. Κάθε model αντιπροσωπεύει μια συλλογή (collection) στη MongoDB και περιλαμβάνει validation rules, default values και relationships μεταξύ των entities.
- **Controllers (Ελεγκτές):** Περιέχουν την επιχειρησιακή λογική της εφαρμογής. Κάθε controller είναι υπεύθυνος για τη διαχείριση των HTTP requests, την επικοινωνία με τη βάση δεδομένων, την επεξεργασία των δεδομένων και την επιστροφή των κατάλληλων responses.
- **Routes (Διαδρομές):** Ορίζουν τα API endpoints και τη δρομολόγηση των αιτημάτων στους αντίστοιχους controllers. Κάθε route file ομαδοποιεί λογικά συναφή endpoints (π.χ. /courses, /users, /analytics).
- **Middleware (Ενδιάμεσο Λογισμικό):** Παρέχει διασταυρούμενες λειτουργίες που εκτελούνται πριν ή μετά τους controllers, όπως authentication, authorization, logging, error handling και CORS configuration.

Αυτός ο διαχωρισμός επιτρέπει την εύκολη επέκταση του συστήματος, τη δοκιμή μεμονωμένων components και την ανεξάρτητη ανάπτυξη διαφορετικών τμημάτων της εφαρμογής.

3.2 Μοντέλα Δεδομένων (Data Models)

3.2.1 Course Schema

Το μοντέλο Course αποτελεί το κεντρικό σχήμα δεδομένων της εφαρμογής και αναπαριστά ένα εκπαιδευτικό μάθημα. Κάθε course προέρχεται από μία εξωτερική πηγή (Coursera,

Microsoft Learn κ.λπ.) και εμπλουτίζεται με μεταδεδομένα που υποστηρίζουν τις λειτουργίες αναζήτησης, φιλτραρίσματος και συστάσεων.

```
1  const courseSchema = new mongoose.Schema({
2    externalId: {
3      type: String,
4      required: true,
5      unique: true,
6      sparse: true
7    },
8    title: { type: String, required: true },
9    description: { type: String, default: "" },
10   language: { type: String, default: "en" },
11   level: { type: String, default: "Beginner" },
12   category: { type: String, default: "General" },
13   keywords: [{ type: String }],
14   source: { type: String },
15   provider: { type: String },
16   url: { type: String },
17   cluster_id: { type: Number, default: -1 },
18   cluster_label: { type: String, default: "" }
19 }, { timestamps: true });
```

Ανάλυση Πεδίων:

- **externalId:** Το μοναδικό αναγνωριστικό του μαθήματος στην εξωτερική πηγή. Το πεδίο είναι `unique` και `sparse`, επιτρέποντας `null values` αλλά διασφαλίζοντας ότι δεν υπάρχουν `duplicates` όταν υπάρχει τιμή. Χρησιμοποιείται για την αποφυγή διπλών εγγραφών κατά τον συγχρονισμό.
- **title, description:** Τα βασικά περιγραφικά πεδία του μαθήματος. Η περιγραφή χρησιμοποιείται εκτενώς στο NLP pipeline του Spark για τον υπολογισμό ομοιότητας.
- **language, level, category:** Μεταδεδομένα για φιλτράρισμα και κατηγοριοποίηση. Έχουν προεπιλεγμένες τιμές για την αντιμετώπιση ελλιπών δεδομένων από τις εξωτερικές πηγές.
- **keywords:** Array από λέξεις-κλειδιά που εξάγονται από το URL ή παρέχονται από την πηγή. Χρησιμοποιούνται για βελτίωση της αναζήτησης.
- **source, provider:** Αναγνωρίζουν την προέλευση του μαθήματος. Ο `provider` είναι η επίσημη ονομασία (π.χ. "Coursera"), ενώ ο `source` μπορεί να είναι το όνομα του `connector`.

- `url`: Το πλήρες URL για πρόσβαση στο μάθημα στην αρχική πλατφόρμα.
- `cluster_id`, `cluster_label`: Προστέθηκαν για την υποστήριξη των αλγορίθμων clustering που εκτελούνται στο Spark layer. Το `cluster_id` είναι ο αριθμητικός αναγνωριστής του cluster, ενώ το `cluster_label` περιέχει την ανθρώπινα αναγνώσιμη ετικέτα (π.χ. "Machine Learning", "Web Development"). Αρχικά έχουν την τιμή -1 και κενό string αντίστοιχα, μέχρι να εκτελεστεί το clustering job.
- `timestamps`: Το option αυτό προσθέτει αυτόματα τα πεδία `createdAt` και `updatedAt`, χρήσιμα για tracking και debugging.

3.2.2 User Schema

Το μοντέλο User διαχειρίζεται τα δεδομένα των χρηστών της εφαρμογής, τις προτιμήσεις τους και το ιστορικό αλληλεπίδρασης με τα μαθήματα. Το schema έχει σχεδιαστεί για να υποστηρίζει personalization features όπως τα αγαπημένα μαθήματα και οι συστάσεις βασισμένες στο ιστορικό προβολής.

```
1  const userSchema = new mongoose.Schema({
2    firstName: { type: String, required: true },
3    lastName: { type: String, required: true },
4    email: { type: String, required: true, unique: true },
5    password: { type: String, required: true },
6    permissionLevel: { type: Number, default: 1 },
7    favoriteCourses: [{
8      type: mongoose.Schema.Types.ObjectId,
9      ref: 'Course'
10   }],
11   recentlyViewed: [{
12     courseId: {
13       type: mongoose.Schema.Types.ObjectId,
14       ref: 'Course'
15     },
16     viewedAt: {
17       type: Date,
18       default: Date.now
19     }
20   }],
21   role: {
22     type: String,
23     enum: ['user', 'admin'],
24     default: 'user'
25   }
```

26 }, { timestamps: true });

Βασικά Χαρακτηριστικά του Schema:

- **Βασικά Στοιχεία Χρήστη:** Τα πεδία `firstName`, `lastName` και `email` είναι υποχρεωτικά. Το `email` είναι `unique` και χρησιμεύει ως `username` για την είσοδο στο σύστημα.
- **Password Security:** Το `password` αποθηκεύεται σε `hashed` μορφή με χρήση `bcrypt` (`salt rounds: 10`). Δεν αποθηκεύεται ποτέ σε `plaintext`.
- **Role-Based Access Control:** Το πεδίο `role` καθορίζει τα δικαιώματα του χρήστη. Υπάρχουν δύο ρόλοι:
 - `'user'`: Κανονικός χρήστης με πρόσβαση σε βασικές λειτουργίες (αναζήτηση, αγαπημένα, προβολή μαθημάτων).
 - `'admin'`: Διαχειριστής με πρόσβαση σε διαχειριστικές λειτουργίες (συγχρονισμός δεδομένων, εκτέλεση Spark jobs, διαχείριση χρηστών).
- **Favorite Courses:** Το πεδίο `favoriteCourses` είναι ένα array από `ObjectIds` που αναφέρονται στη συλλογή `Course`. Χρησιμοποιείται `Mongoose populate()` για να ανακτηθούν τα πλήρη `course objects` όταν χρειάζεται.
- **Recently Viewed:** Το πεδίο `recentlyViewed` αποθηκεύει τα τελευταία 30 μαθήματα που προέβαλε ο χρήστης. Κάθε `entry` περιέχει:
 - `courseId`: Reference στο μάθημα
 - `viewedAt`: Timestamp της προβολήςΤο array διατηρείται ταξινομημένο με τα πιο πρόσφατα πρώτα και περιορίζεται στα 30 `items` για αποδοτικότητα.
- **Permission Level:** Το πεδίο `permissionLevel` (default: 1) επιτρέπει μελλοντική επέκταση με περισσότερους ρόλους και `granular permissions`.

3.2.3 Recommendation Schema

Το μοντέλο Recommendation αποθηκεύει τα αποτελέσματα των αλγορίθμων συστάσεων που εκτελούνται στο Spark layer. Κάθε εγγραφή αναπαριστά μια σύσταση: δηλαδή ότι το μάθημα B συνιστάται για όποιον ενδιαφέρεται για το μάθημα A.

```
1  const RecommendationSchema = new mongoose.Schema({
2    source_course_id: {
3      type: String,
4      required: true,
5      index: true
6    },
7    recommended_course_id: {
8      type: String,
9      required: true
10   },
11   score: Number,
12   rank: Number
13 }, {
14   collection: 'course_recommendations'
15 });
```

Δομή και Λειτουργία:

- **Source Course:** Το πεδίο `source_course_id` είναι το μάθημα για το οποίο γίνεται η σύσταση. Έχει index για γρήγορη αναζήτηση όλων των συστάσεων για ένα συγκεκριμένο μάθημα.
- **Recommended Course:** Το πεδίο `recommended_course_id` είναι το μάθημα που συνιστάται.
- **Similarity Score:** Το πεδίο `score` περιέχει το σκορ ομοιότητας (συνήθως από 0 έως 1), υπολογισμένο από τον αλγόριθμο του Spark (cosine similarity, Jaccard distance κ.λπ.). Όσο μεγαλύτερο το score, τόσο πιο παρόμοια είναι τα μαθήματα.
- **Rank:** Το πεδίο `rank` υποδεικνύει τη σειρά κατάταξης της σύστασης (1 = καλύτερη σύσταση, 2 = δεύτερη καλύτερη κ.ο.κ.). Το Spark layer κρατάει μόνο τις top-5 συστάσεις για κάθε μάθημα.
- **Collection Name:** Το option `collection: 'course_recommendations'` εξασφαλίζει ότι το Mongoose θα χρησιμοποιήσει το συγκεκριμένο όνομα collection στη MongoDB, αντί του default pluralized name.

Παράδειγμα Χρήσης: Αν το μάθημα "Introduction to Python" έχει `_id: "abc123"`, μια εγγραφή στο Recommendation collection μπορεί να είναι:

```
{
  source_course_id: "abc123",
  recommended_course_id: "def456",
  score: 0.87,
  rank: 1
}
```

Αυτό σημαίνει ότι το μάθημα με id "def456" (π.χ. "Data Structures in Python") είναι η πρώτη σύσταση για όποιον κοιτάζει το "Introduction to Python", με similarity score 0.87.

3.3 Authentication και Authorization

3.3.1 JWT-Based Authentication

Το σύστημα χρησιμοποιεί **JSON Web Tokens (JWT)** για την ταυτοποίηση των χρηστών. Τα JWT είναι ένα ανοιχτό πρότυπο (RFC 7519) που επιτρέπει την ασφαλή μεταφορά πληροφοριών μεταξύ δύο μερών ως ένα JSON object. Τα tokens είναι digitally signed και περιέχουν claims (δηλώσεις) σχετικά με τον χρήστη.

Ροή Λειτουργίας:

1. Ο χρήστης συνδέεται με email/password
2. Το backend επαληθεύει τα credentials και δημιουργεί ένα JWT
3. Το JWT αποθηκεύεται στη MongoDB (collection Token) και στέλνεται στον client
4. Ο client στέλνει το JWT σε κάθε επόμενο request (μέσω cookie ή Authorization header)
5. Το middleware `verifyToken` ελέγχει την εγκυρότητα του token πριν επιτρέψει την πρόσβαση

```
1 const verifyToken = async (req, res, next) => {
2   let token = req.cookies.token;
3   const authHeader = req.headers['authorization'];
4
```

```
5     if (!token && authHeader &&
6         authHeader.startsWith('Bearer ')) {
7         token = authHeader.split(' ')[1];
8     }
9
10    if (!token) {
11        return res.status(401).json({
12            message: "Access Denied: No Token Provided"
13        });
14    }
15
16    try {
17        const tokenExists = await Token.findOne({ token });
18        if (!tokenExists) {
19            return res.status(401).json({
20                message: "Token is no longer valid"
21            });
22        }
23
24        const verified = jwt.verify(
25            token,
26            process.env.JWT_SECRET
27        );
28
29        req.userID = verified.id;
30        req.userRole = verified.role;
31        next();
32    } catch (error) {
33        res.status(401).json({
34            message: "Token expired or invalid"
35        });
36    }
37 };
```

Λεπτομερής Ανάλυση του Middleware:

1. **Εξαγωγή Token:** Το middleware υποστηρίζει δύο τρόπους αποστολής του token:
 - **HTTP-only Cookie:** Πιο ασφαλής για web browsers, καθώς το token δεν είναι προσβάσιμο από JavaScript του client, προστατεύοντας από XSS attacks.
 - **Authorization Header:** Για mobile apps, API clients και testing tools που δεν υποστηρίζουν cookies. Το format είναι: `Authorization: Bearer <token>`.
2. **Token Blacklisting:** Πριν την επαλήθευση, ελέγχεται αν το token υπάρχει στο Token collection της MongoDB. Αυτό υλοποιεί έναν μηχανισμό blacklisting:

- Κατά την αποσύνδεση (signout), το token διαγράφεται από το collection.
 - Έτσι, ακόμα και αν ένας εισβολέας αποκτήσει πρόσβαση σε ένα token, μετά την αποσύνδεση το token γίνεται άκυρο.
3. **JWT Verification:** Χρησιμοποιείται η βιβλιοθήκη jsonwebtoken για την επαλήθευση του signature και την αποκωδικοποίηση του payload. Το JWT_SECRET από το environment χρησιμεύει ως το κλειδί υπογραφής.
4. **Request Enrichment:** Μετά την επιτυχή επαλήθευση, το middleware προσθέτει τα req.userID και req.userRole στο request object, τα οποία χρησιμοποιούνται από τους controllers για personalization και authorization.
5. **Error Handling:** Διαφορετικά error messages για διαφορετικά σενάρια (missing token, invalid token, expired token) βοηθούν στο debugging και την καλύτερη εμπειρία χρήστη.

3.3.2 Role-Based Authorization

Το middleware isAdmin περιορίζει την πρόσβαση σε ευαίσθητες λειτουργίες:

```
1 const isAdmin = (req, res, next) => {  
2   if (req.userRole === 'admin') {  
3     next();  
4   } else {  
5     res.status(403).json({  
6       message: "Access Denied: Admins Only"  
7     });  
8   }  
9 };
```

Το middleware εφαρμόζεται σε endpoints που αφορούν τη διαχείριση του συστήματος, όπως ο συγχρονισμός δεδομένων και η εκτέλεση Spark jobs.

3.4 CRUD Operations και API Endpoints

3.4.1 Ανάκτηση Μαθημάτων με Φίλτρα

Η συνάρτηση getAllCourses() υποστηρίζει σύνθετα φίλτρα:

```
1  const getAllCourses = async (req, res) => {
2    try {
3      const {
4        page = 1,
5        limit = 10,
6        provider,
7        title,
8        language,
9        level,
10       category
11     } = req.query;
12
13     const query = {};
14     if (provider) query.provider = provider;
15     if (title) query.title = {
16       $regex: title,
17       $options: 'i'
18     };
19     if (language) query.language = language;
20     if (level) query.level = level;
21     if (category) query.category = category;
22
23     const courses = await Course.find(query)
24       .sort({ createdAt: -1 })
25       .exec();
26
27     const count = await Course.countDocuments(query);
28
29     res.json({
30       courses,
31       totalPages: Math.ceil(count / limit),
32       currentPage: Number(page),
33       totalResults: count
34     });
35   } catch (error) {
36     res.status(500).json({
37       error: 'Failed to fetch courses'
38     });
39   }
40 };
```

Χαρακτηριστικά:

- Regex-based filtering στον τίτλο για fuzzy search.
- Ακριβής αντιστοίχιση για τα υπόλοιπα πεδία (provider, language, level, category).
- Pagination support με υπολογισμό του συνολικού αριθμού σελίδων.

3.4.2 Σύστημα Συστάσεων

Η συνάρτηση `getSimilarCourses()` ανακτά συναφή μαθήματα:

```
1  const getSimilarCourses = async (req, res) => {  
2    try {  
3      const { id } = req.params;  
4  
5      const recommendations = await Recommendation.find({  
6        $or: [  
7          { source_course_id: id },  
8          { recommended_course_id: id }  
9        ]  
10     })  
11     .sort({ rank: 1 })  
12     .limit(5)  
13     .lean();  
14  
15     if (!recommendations.length) return res.json([]);  
16  
17     const recommendedCourseIds = recommendations.map(r =>  
18       r.recommended_course_id === id  
19       ? r.source_course_id  
20       : r.recommended_course_id  
21     );  
22  
23     const courses = await Course.find({  
24       _id: { $in: recommendedCourseIds }  
25     }).lean();  
26  
27     const results = recommendations.map(rec => {  
28       const targetId = rec.recommended_course_id === id  
29       ? rec.source_course_id  
30       : rec.recommended_course_id;  
31       const courseData = courses.find(  
32         c => c._id.toString() === targetId  
33       );  
34       return courseData  
35       ? { ...courseData, score: rec.score }  
36       : null;  
37     }).filter(Boolean);  
38  
39     res.json(results);  
40   } catch (error) {  
41     res.status(500).json({  
42       error: 'Internal Server Error'  
43     });  
44   }  
}
```

45 };

Τεχνική Υλοποίηση:

- Χρήση `$or` operator για συμμετρική αναζήτηση (το μάθημα μπορεί να είναι source ή recommended).
- Δύο ξεχωριστά queries: ένα για τις συστάσεις και ένα για τα course objects.
- Join των αποτελεσμάτων σε application level για προσθήκη του similarity score.

3.4.3 Διαχείριση Αγαπημένων Μαθημάτων

Οι χρήστες μπορούν να προσθέσουν μαθήματα στα αγαπημένα τους:

```
1  const postFavoriteCourse = async (req, res) => {
2    try {
3      const { id } = req.params;
4      const userID = req.userID;
5
6      const updatedUser = await User.findByIdAndUpdate(
7        userID,
8        { $addToSet: { favoriteCourses: id } },
9        { new: true }
10     ).populate('favoriteCourses');
11
12     if (!updatedUser) {
13       return res.status(404).json({
14         error: 'User not found.'
15       });
16     }
17
18     res.status(200).json({
19       favoriteCourses: updatedUser.favoriteCourses
20     });
21   } catch (error) {
22     res.status(500).json({
23       error: 'Failed to add course to favorites.'
24     });
25   }
26 };
```

Ο operator `$addToSet` εξασφαλίζει ότι δεν θα δημιουργηθούν duplicates στη λίστα αγαπημένων, ακόμα και αν ο χρήστης κάνει επανειλημμένα κλικ.

3.4.4 Ιστορικό Προβολής

Το σύστημα διατηρεί ιστορικό των πρόσφατα προβεβλημένων μαθημάτων:

```
1  const postRecentlyViewedCourse = async (req, res) => {
2    try {
3      const { id } = req.params;
4      const userID = req.userID;
5
6      // Αφαίρεση προηγούμενης εγγραφής
7      await User.updateOne(
8        { _id: userID },
9        { $pull: { recentlyViewed: { courseId: id } } }
10     );
11
12     const updatedUser = await User.findByIdAndUpdate(
13       userID,
14       {
15         $push: {
16           recentlyViewed: {
17             $each: [{
18               courseId: id,
19               viewedAt: new Date()
20             }],
21             $position: 0,
22             $slice: 30
23           }
24         },
25         { new: true }
26       ).populate('recentlyViewed.courseId');
27
28     res.status(200).json({
29       recentlyViewed: updatedUser.recentlyViewed
30     });
31   } catch (error) {
32     res.status(500).json({
33       error: 'Failed to update recently viewed.'
34     });
35   }
36 }
37 };
```

Λογική Λειτουργίας:

1. Αφαίρεση προηγούμενης εγγραφής για το ίδιο μάθημα (ανανέωση timestamp).
2. Προσθήκη στην αρχή της λίστας (\$position: 0).

3. Διατήρηση μόνο των 30 πιο πρόσφατων (`$slice: 30`).

3.5 Analytics και Στατιστικά

Η συνάρτηση `getStats()` παρέχει αναλυτικά στατιστικά:

```
1  const getStats = async (req, res) => {
2    try {
3      const courses = await Course.find({});
4      const totalCourses = courses.length;
5
6      // Στατιστικά ανά πηγή
7      const providerCount = {};
8      courses.forEach(course => {
9        const provider = course.provider ||
10          course.source || 'Άγνωστη';
11        providerCount[provider] =
12          (providerCount[provider] || 0) + 1;
13      });
14
15      const bySource = Object.entries(providerCount)
16        .map(([name, count]) => ({
17          name,
18          count,
19          percentage: Math.round(
20            (count / totalCourses) * 100
21          )
22        }))
23        .sort((a, b) => b.count - a.count);
24
25      // Στατιστικά ανά γλώσσα
26      const languageMap = {
27        'en': 'Αγγλικά',
28        'gr': 'Ελληνικά',
29        'el': 'Ελληνικά',
30        'es': 'Ισπανικά',
31        'fr': 'Γαλλικά'
32      };
33
34      const languageCount = {};
35      courses.forEach(course => {
36        const lang = course.language || 'en';
37        const langName =
38          languageMap[lang.toLowerCase()] ||
39          lang.toUpperCase();
40        languageCount[langName] =
41          (languageCount[langName] || 0) + 1;
```

```
42     });
43
44     const byLanguage = Object.entries(languageCount)
45       .map(([name, count]) => ({
46         name,
47         count,
48         percentage: Math.round(
49           (count / totalCourses) * 100
50         )
51       }))
52       .sort((a, b) => b.count - a.count);
53
54     res.json({
55       totalCourses,
56       bySource,
57       byLanguage,
58       uniqueSources: new Set(
59         courses.map(c => c.source || c.provider)
60           .filter(Boolean)
61         ).size,
62       lastSync: new Date().toISOString()
63     });
64   } catch (error) {
65     res.status(500).json({
66       error: 'Failed to calculate analytics'
67     });
68   }
69 };
```

3.6 Ενσωμάτωση με Spark Layer

Το API παρέχει endpoints για την εκτέλεση Spark jobs:

```
1  const runSparkJob = async (req, res) => {
2    const scriptPath = "../spark-layer/jobs/recommender.py";
3    const recommender_option = req.params.option || '0';
4
5    const pythonProcess = spawn(
6      'python3',
7      [scriptPath, recommender_option],
8      { shell: true }
9    );
10
11    let output = "";
12    let errorOutput = "";
13
14    pythonProcess.stdout.on('data', (data) => {
```

```
15     output += data.toString();
16   });
17
18   pythonProcess.stderr.on('data', (data) => {
19     errorOutput += data.toString();
20     console.error("Spark Log:", data.toString());
21   });
22
23   const timeout = setTimeout(() => {
24     pythonProcess.kill();
25     if (!res.headersSent) {
26       res.status(408).json({
27         message: "Spark Job Timeout"
28       });
29     }
30   }, 180000); // 3 minute timeout
31
32   pythonProcess.on('close', (code) => {
33     clearTimeout(timeout);
34     if (code !== 0) {
35       if (!res.headersSent) {
36         return res.status(500).json({
37           message: "Spark Error",
38           error: errorOutput
39         });
40       }
41     }
42     if (!res.headersSent) {
43       res.status(200).json({
44         message: "Success",
45         details: output
46       });
47     }
48   });
49 };
```

Χαρακτηριστικά:

- Asynchronous execution με child processes.
- Timeout protection για μακροχρόνιες εργασίες (3 λεπτά).
- Stream handling για stdout και stderr.
- Error propagation στο frontend για debugging.

4 Harvester (Data Connectors)

Το σύστημα συλλογής δεδομένων (data harvesting layer) αποτελεί την εισόδου δεδομένων της εφαρμογής. Είναι υπεύθυνο για τη συστηματική και αυτόματη συγκέντρωση εκπαιδευτικού περιεχομένου από διαφορετικές εξωτερικές πηγές (πλατφόρμες online learning), τη μετατροπή τους σε ενιαίο format και την αποθήκευσή τους στη βάση δεδομένων.

Η αρχιτεκτονική των connectors έχει σχεδιαστεί με γνώμονα:

- **Επεκτασιμότητα:** Δυνατότητα προσθήκης νέων πηγών χωρίς αλλαγές στον core κώδικα.
- **Ομοιομορφία:** Κάθε connector ακολουθεί το ίδιο interface, ανεξάρτητα από τη δομή του εξωτερικού API.
- **Αντοχή σε λάθη:** Η αποτυχία ενός connector δεν επηρεάζει τους υπόλοιπους.
- **Διαχείριση Ετερογενών Δεδομένων:** Κάθε πηγή έχει διαφορετική δομή δεδομένων που πρέπει να κανονικοποιηθεί.

4.1 Αρχιτεκτονική Connectors

Κάθε connector υλοποιείται ως ένα ανεξάρτητο JavaScript module (.js file) που τοποθετείται στο directory `services/connectors/`. Αυτή η modular προσέγγιση επιτρέπει την εύκολη προσθήκη, τροποποίηση ή αφαίρεση πηγών δεδομένων χωρίς να επηρεάζεται ο υπόλοιπος κώδικας.

Κάθε connector εξάγει (exports) δύο υποχρεωτικές συναρτήσεις:

1. `getCourses()`:

- **Ρόλος:** Αναλαμβάνει την επικοινωνία με το εξωτερικό API.
- **Λειτουργία:** Διαχειρίζεται authentication (αν απαιτείται), κάνει τα HTTP requests και επιστρέφει τα ακατέργαστα δεδομένα όπως έρχονται από την πηγή.
- **Επιστροφή:** Ένα JavaScript object ή array με τα raw data από το API.

2. `normalize(data)`:

- **Ρόλος:** Μετατρέπει τα ετερογενή δεδομένα σε ενιαίο σχήμα.

- **Λειτουργία:** Παίρνει τα raw data από το `getCourses()` και τα χαρτογραφεί στα πεδία του Course schema.
- **Επιστροφή:** Ένα array από course objects έτοιμα για αποθήκευση στη MongoDB.

Γιατί αυτός ο διαχωρισμός;

Η αρχιτεκτονική αυτή ακολουθεί την αρχή του *separation of concerns*:

- Η λογική επικοινωνίας με το API (authentication, pagination, error handling) είναι ξεχωριστή από τη λογική μετασχηματισμού δεδομένων.
- Αν αλλάξει η δομή του εξωτερικού API, χρειάζεται να τροποποιηθεί μόνο η `normalize()`.
- Αν αλλάξει το authentication mechanism, τροποποιείται μόνο η `getCourses()`.
- Το testing γίνεται ευκολότερο καθώς κάθε συνάρτηση μπορεί να δοκιμαστεί ανεξάρτητα.

4.2 Connector για Coursera

Ο connector της Coursera αξιοποιεί το επίσημο Partner API της πλατφόρμας, το οποίο απαιτεί OAuth 2.0 authentication με client credentials flow.

4.2.1 Διαδικασία Ταυτοποίησης

Η συνάρτηση `fetchFromRepoA()` υλοποιεί την ταυτοποίηση:

```
1  async function fetchFromRepoA() {
2    const auth = Buffer.from(
3      `${process.env.REPO_A_KEY}:${process.env.REPO_A_SECRET}`
4    ).toString('base64');
5
6    const response = await axios.post(
7      process.env.REPO_A_URL,
8      'grant_type=client_credentials',
9      {
10       headers: {
11         'Authorization': `Basic ${auth}`,
12         'Content-Type':
13           'application/x-www-form-urlencoded'
14       }
15     }
```

```
15     }
16   );
17
18   return response.data.access_token;
19 }
```

Το access token που επιστρέφεται χρησιμοποιείται για την εξουσιοδότηση των επακόλουθων αιτημάτων προς το API.

4.2.2 Συλλογή Δεδομένων Μαθημάτων

```
1  async function getCourses() {
2    try {
3      const token = await fetchFromRepoA();
4      const response = await axios.get(
5        process.env.REPO_A_COURSES_URL,
6        {
7          headers: {
8            'Authorization': `Bearer ${token}`
9          },
10         params: {
11           fields: 'description,difficultyLevel,
12                 primaryLanguages,slug,domainTypes',
13           includes: 'description',
14           limit: 100
15         }
16       });
17     };
18     return response.data;
19   } catch (error) {
20     console.error("Connector Error:",
21       error.response ?
22         error.response.data : error.message);
23     throw error;
24   }
25 }
```

Τα parameters που περιλαμβάνονται στο αίτημα επιτρέπουν τον προσδιορισμό των πεδίων που θα επιστραφούν, μειώνοντας το όγκο των δεδομένων.

4.2.3 Ομαλοποίηση Δεδομένων

Η συνάρτηση `normalize()` μετασχηματίζει τη δομή του API:

```
1 function normalize(data) {
2   if (!data || !data.elements) return [];
3
4   return data.elements.map(item => ({
5     externalId: item.id,
6     title: item.name,
7     description: item.description ||
8       "No description available",
9     provider: 'Coursera',
10    url: `https://www.coursera.org/learn/${item.slug}`,
11    language: (item.primaryLanguages &&
12      item.primaryLanguages.length > 0)
13      ? item.primaryLanguages[0] : 'en',
14    level: item.difficultyLevel || 'Beginner',
15    category: (item.domainTypes &&
16      item.domainTypes.length > 0)
17      ? item.domainTypes[0].domainId : 'General',
18    keywords: item.slug ? item.slug.split('-') : []
19  }));
20 }
```

Διαδικασία Ομαλοποίησης:

- Επαναχαρατογράφηση πεδίων (π.χ. `item.name` → `title`).
- Προεπιλεγμένες τιμές για προαιρετικά πεδία.
- Εξαγωγή keywords από το URL slug.
- Δημιουργία πλήρους URL για το μάθημα.

4.3 Connector για Microsoft Learn

Ο connector του Microsoft Learn συλλέγει δύο τύπους περιεχομένου: modules και learning paths.

4.3.1 Συλλογή Δεδομένων

```
1 async function getCourses() {
2   try {
3     const response = await axios.get(
4       process.env.REPO_B_URL
5     );
```

```
6     return {
7         modules: response.data.modules || [],
8         learningPaths: response.data.learningPaths || []
9     };
10 } catch (error) {
11     console.error("Connector Error:",
12         error.response ?
13         error.response.data : error.message);
14     throw error;
15 }
16 }
```

Το Microsoft Learn API παρέχει ένα ανοιχτό endpoint που δεν απαιτεί authentication, απλοποιώντας τη διαδικασία συλλογής.

4.3.2 Ομαλοποίησης Ετερογενών Δεδομένων

```
1 function normalize(data) {
2     const all_items = [
3         ...(data.modules || []),
4         ...(data.learningPaths || [])
5     ];
6
7     if(all_items.length === 0) return [];
8
9     return all_items.map(item => ({
10         externalId: item.uid,
11         title: item.title,
12         description: item.summary ||
13             "No description available",
14         provider: 'Microsoft Learn',
15         url: item.url,
16         language: item.locale || 'en',
17         level: (item.levels && item.levels.length > 0)
18             ? item.levels[0] : 'Beginner',
19         category: (item.roles && item.roles.length > 0)
20             ? item.roles[0] : 'Technical',
21         keywords: item.products || [],
22         source: 'Microsoft Learn'
23     }));
24 }
```

Η συγχώνευση των modules και learning paths σε ενιαίο array επιτρέπει την ομοιόμορφη επεξεργασία και των δύο τύπων περιεχομένου.

4.4 Δυναμική Ανακάλυψη Connectors

Το σύστημα υποστηρίζει δυναμική προσθήκη νέων connectors χωρίς αλλαγές στον κεντρικό κώδικα:

```
1  const getAvailableSources = async (req, res) => {
2    try {
3      const connectorsPath = path.join(
4        __dirname,
5        '../services/connectors'
6      );
7
8      if(!fs.existsSync(connectorsPath)) {
9        return res.status(404).json({
10          error: 'Connectors directory not found'
11        });
12      }
13
14      const files = fs.readdirSync(connectorsPath);
15      const sources = files
16        .filter(file => file.endsWith('.js'))
17        .map(file => file.replace('.js', ''));
18
19      res.json({ sources });
20    } catch (error) {
21      res.status(500).json({
22        error: 'Failed to fetch sources'
23      });
24    }
25  };
```

Αυτή η προσέγγιση επιτρέπει την προσθήκη νέων πηγών δεδομένων με απλή δημιουργία ενός νέου αρχείου JavaScript στο κατάλληλο directory.

4.5 Συγχρονισμός Δεδομένων

4.5.1 Συγχρονισμός Μεμονωμένης Πηγής

```
1  const syncSource = async (req, res) => {
2    try {
3      const { source } = req.params;
4
5      let connector;
6      try {
```

```
7         connector = require(  
8             `../services/connectors/${source}.js`  
9         );  
10    } catch (err) {  
11        return res.status(400).json({  
12            error: `Provider ${source} not supported.`  
13        });  
14    }  
15  
16    const data = await connector.getCourses();  
17    const normalized = connector.normalize(data);  
18  
19    const bulkOps = normalized.map(course => ({  
20        replaceOne: {  
21            filter: { externalId: course.externalId },  
22            replacement: {  
23                ...course,  
24                source: course.source  
25            },  
26            upsert: true  
27        }  
28    }));  
29  
30    if (bulkOps.length > 0) {  
31        await Course.bulkWrite(bulkOps);  
32    }  
33  
34    return res.json({  
35        message: "Sync successful",  
36        source,  
37        count: normalized.length  
38    });  
39    } catch (error) {  
40        res.status(500).json({ error: error.message });  
41    }  
42    };
```

Πλεονεκτήματα του `bulkWrite()`:

- Αποδοτική εκτέλεση με ελάχιστες round-trips στη βάση.
- Upsert semantics (insert ή update ανάλογα με την ύπαρξη).
- Διατήρηση του `externalId` ως unique key.

4.5.2 Ομαδικός Συγχρονισμός

Η συνάρτηση `syncAllSources()` εκτελεί συγχρονισμό όλων των διαθέσιμων connectors:

```
1  const syncAllSources = async (req, res) => {
2    try {
3      const connectorsPath = path.join(
4        __dirname,
5        '../services/connectors'
6      );
7      const files = fs.readdirSync(connectorsPath);
8      const sources = files
9        .filter(file => file.endsWith('.js'))
10       .map(file => file.replace('.js', ''));
11
12      let totalAdded = 0;
13      const results = [];
14
15      for (const source of sources) {
16        try {
17          const connector = require(
18            `../services/connectors/${source}.js`
19          );
20          const data = await connector.getCourses();
21          const normalized = connector.normalize(data);
22
23          const bulkOps = normalized.map(course => ({
24            replaceOne: {
25              filter: {
26                externalId: course.externalId
27              },
28              replacement: { ...course },
29              upsert: true
30            }
31          }));
32
33          if (bulkOps.length > 0) {
34            await Course.bulkWrite(bulkOps);
35            totalAdded += normalized.length;
36            results.push({
37              source,
38              status: 'success',
39              count: normalized.length
40            });
41          }
42        } catch (err) {
43          results.push({
```

```
44         source,  
45         status: 'failed',  
46         error: err.message  
47     });  
48 }  
49 }  
50  
51 res.json({  
52     message: "Global sync completed",  
53     added: totalAdded,  
54     details: results  
55 });  
56 } catch (error) {  
57     res.status(500).json({  
58         error: 'Global sync failed: ' + error.message  
59     });  
60 }  
61 };
```

Η διαδικασία περιλαμβάνει error handling ανά connector, ώστε η αποτυχία ενός να μην επηρεάζει τους υπόλοιπους.

5 Spark Layer

5.1 System Architecture

Το σύστημα σε pySpark που σχεδιάστηκε και υλοποιήθηκε ακολουθεί την εξής ροή δεδομένων (Pipeline):

1. Data Ingestion: Ανάγνωση δεδομένων από το collection courses της MongoDB μέσω Spark
2. Preprocessing (NLP): Καθαρισμός κειμένου (Title + Description + Keywords με SparkNLP
3. Vectorization: Μετατροπή κειμένου σε αριθμητικά διανύσματα (TF-IDF)
4. Modeling:
 - Clustering (LDA) για θεματική ομαδοποίηση.
 - Similarity Calculation (Cosine Similarity / LSH).

5. Data Export: Εγγραφή των συστάσεων πίσω στη MongoDB (course_recommendations/courses για τα πεδία cluster_id και cluster_label).

5.2 Μεθοδολογία & Αλγόριθμοι

5.2.1 NLP Pipeline

Βρίσκεται το `jobs/utills/` και αποτελείται από τα εξής βήματα

1. `DocumentAssembler` Ετοιμάζει τα δεδομένα, τα μετατρέπει σε μορφή document, ένα format που μπορεί να διαχειριστεί η βιβλιοθήκη SparkNLP
2. `Tokenizer` Σπάει το αρχικό κείμενο στις λέξεις (tokens) από τις οποίες αποτελείται,
3. `Normalizer` Καθαρίζει τα tokens αφαιρώντας σημεία στίξης, σύμβολα και ειδικούς χαρακτήρες με χρήση regex, διατηρώντας συνήθως μόνο τους αλφαριθμητικούς χαρακτήρες ώστε να απομείνουν οι καθαρές λέξεις για περαιτέρω ανάλυση.
4. `StopWordsCleaner` Αφαιρεί τα tokens τα οποία δεν έχουν σημασιολογικό περιεχόμενο, κοινές λέξεις όπως τα "and", "or" etc. Μεταξύ αυτών προστέθηκαν και κάποιες έξτρα όπως οι λέξεις "courses", "introduction", "student" etc.
5. `LemmatizerModel/Stemmer` Εναλλακτικές τεχνικές για να ανάγουμε μία λέξη στην αρχική της μορφή (ρίζα). Ο Stemmer είναι γρηγορότερος καθώς χρησιμοποιεί προκαθορισμένους κανόνες για την κοπή των καταλήξεων, αλλά μπορεί να καταλήξει σε μη υπαρκτές λέξεις (π.χ. το "studies" να γίνει "studi"). Αντίθετα, το Lemmatizer-Model χρησιμοποιεί λεξικό και μορφολογική ανάλυση για να βρει το πραγματικό λήμμα της λέξης (π.χ. το "better" να γίνει "good"), προσφέροντας μεγαλύτερη ακρίβεια στη σημασιολογική ανάλυση. Χρησιμοποιείται pretrained εκδοχή του Lemmatizer.
6. `Finisher` Αποτελεί το τελευταίο στάδιο του NLP Pipeline, το οποίο μετατρέπει τα δεδομένα από την εσωτερική μορφή "Annotations" της SparkNLP σε απλά Spark SQL Arrays. Αυτό γίνεται ώστε τα αποτελέσματα του Pipeline να είναι συμβατά με τις κλασικές βιβλιοθήκες μηχανικής μάθησης του SparkML (όπως ο CountVectorizer).

5.2.2 TF-IDF

Η TF-IDF (Term Frequency–Inverse Document Frequency) είναι μια στατιστική μέθοδος που χρησιμοποιείται σε εφαρμογές NLP για την αξιολόγηση της σημασίας μιας λέξης για ένα έγγραφο σε σχέση με μια μεγαλύτερη συλλογή εγγράφων (corpus).

Το TF-IDF αποτυπώνει δύο βασικές ιδέες σχετικά με τη συνάφεια ενός όρου με ένα έγγραφο. Πρώτον, θα περιμέναμε ότι όσο πιο συχνά εμφανίζεται ένας όρος σε ένα έγγραφο, τόσο πιο σημαντικός είναι για το έγγραφο αυτό (TF). Δεύτερον, δεν είναι όλοι οι όροι εξίσου σημαντικοί σε καθολικό επίπεδο. Είναι πιο σημαντικό να συναντήσετε μια λέξη που εμφανίζεται σπάνια σε ολόκληρο το corpus παρά μια λέξη που εμφανίζεται στα περισσότερα έγγραφα έτσι, η μετρική χρησιμοποιεί το αντίστροφο της εμφάνισης της λέξης σε έγγραφα στο πλήρες corpus (IDF).

Η συχνότητα των λέξεων σε ένα σώμα κειμένων τείνει να κατανέμεται εκθετικά. Μία κοινή λέξη εμφανίζεται εκθετικά περισσότερες φορές από μία σπάνια λέξη. Το IDF είναι μία μετρική που δίνει σε αυτές στις σπάνιες λέξεις τεράστιο βάρος σε σχέση με τις συχνές, τις οποίες πρακτικά θα τις αγνοούσε. Αυτό διατυπώνεται ως εξής:

$$\text{idf}(t, D) = \log \frac{N}{n_t} \quad (1)$$

Πρόκειται τεχνική βασισμένη στο bag-of-words model. Δηλαδή έχει το μειονέκτημα ότι δεν δίνει καμία σημασία στη διάταξη των λέξεων, στη δομή της πρότασης ή στις αρνήσεις. Επίσης το μοντέλο αδυνατεί να αντιμετωπίσει το γεγονός ότι η ίδια λέξη μπορεί να έχει πολλές σημασίες ανάλογα σε διαφορετικά contexts. Μία εναλλακτική που θα λάμβανε υπόψη τα παραπάνω θα ήταν η χρήση embeddings, αλλά αυτό ξεπερνάει τα πλαίσια της εργασίας.

Το TF υλοποιείται από τον CountVectorizer και το IDF από το την κλάση του ίδιου ονόματος. Χρησιμοποιήθηκαν οι παράμετροι minDF για τον αποκλεισμό πολύ σπάνιων όρων και maxDF για την αφαίρεση όρων που εμφανίζονται σε πάνω από το 60% των εγγράφων, λειτουργώντας ως δυναμικό stop-word filter.

```
1 cv = CountVectorizer(inputCol="tokens", outputCol="raw_features",
2   minDF=10, maxDF=0.6, vocabSize=20000)
3 cv_model = cv.fit(valid_tokens_df)
4 vectorized_tokens = cv_model.transform(valid_tokens_df)
5
6 idf = IDF(inputCol="raw_features", outputCol="features")
7 idf_model = idf.fit(vectorized_tokens)
```



```
8 final_vectorized_df = idf_model.transform(vectorized_tokens) \
9     .drop("raw_features")
```

Η παραπάνω λογική βρίσκεται στην συνάρτηση `clean_and_prepare_features` του `nlp_pipeline.py`.

5.2.3 Αναπαράσταση Δεδομένων: Document-Term Matrix

Το τελικό αποτέλεσμα της διαδικασίας TF-IDF είναι ένα Document-Term Matrix. Σε αυτή τη δομή:

- Κάθε **γραμμή** αντιπροσωπεύει ένα μάθημα.
- Κάθε **στήλη** αντιπροσωπεύει μια μοναδική λέξη από το λεξιλόγιο των 20.000 όρων.
- Η τιμή σε κάθε κελί είναι το βάρος TF-IDF που υποδηλώνει τη σημασία της λέξης για το συγκεκριμένο μάθημα.

Λόγω της φύσης των κειμένων, η συντριπτική πλειοψηφία των κελιών περιέχει την τιμή μηδέν. Για τη βελτιστοποίηση της μνήμης και της υπολογιστικής ισχύος, το Spark χρησιμοποιεί τη δομή `SparseVector`. Αντί για έναν πλήρη πίνακα (Dense Matrix), αποθηκεύονται μόνο:

1. Το συνολικό μέγεθος του λεξιλογίου.
2. Οι δείκτες (indices) των στηλών που έχουν μη μηδενική τιμή.
3. Οι αντίστοιχες τιμές TF-IDF για αυτούς τους δείκτες.

Αυτή η προσέγγιση επιτρέπει στον αλγόριθμο να εκτελεί πράξεις γραμμικής άλγεβρας (όπως ο υπολογισμός cosine similarity) εξαιρετικά γρήγορα, αγνοώντας τα μηδενικά στοιχεία.

5.3 Στρατηγικές Παραγωγής Συστάσεων (Recommendation Scenarios)

Στο σύστημα υλοποιήθηκαν (στο `recommender.py`) τέσσερις διαφορετικές προσεγγίσεις για τον υπολογισμό της ομοιότητας μεταξύ των μαθημάτων, καθεμία με διαφορετικά πλεονεκτήματα ως προς την ακρίβεια και την ταχύτητα.

5.3.1 Σενάριο 0: Exact Cosine Similarity (Ground Truth)

Το σενάριο αυτό αποτελεί το "χρυσό πρότυπο" για την αξιολόγηση των συστάσεων. Χρησιμοποιεί cosine similarity, η οποία υπολογίζει τη γωνία μεταξύ δύο διανυσμάτων TF-IDF στον πολυδιάστατο χώρο με τον εξής τύπο:

$$\text{similarity} = \cos(\theta) = \frac{\mathbf{A} \cdot \mathbf{B}}{|\mathbf{A}| |\mathbf{B}|} \quad (2)$$

Τεχνική Υλοποίηση:

- Τα διανύσματα κανονικοποιούνται (L2 Normalization) έτσι ώστε το μήκος τους να είναι 1.
- Ο υπολογισμός γίνεται μέσω κατανεμημένου πολλαπλασιασμού πινάκων (AA^T) με τη χρήση της κλάσης `BlockMatrix` του Spark.
- **Πλεονέκτημα:** Μέγιστη μαθηματική ακρίβεια.
- **Μειονέκτημα:** Υψηλό υπολογιστικό κόστος $O(N^2)$, το οποίο όμως διαχειρίζεται το Spark μέσω παραλληλισμού.

5.3.2 Σενάριο 1: Approximate k-NN (MinHash LSH)

Για περιπτώσεις πολύ μεγάλων δεδομένων, η χρήση προσέγγισης είναι απαραίτητη λόγω του απαγορευτικού $O(N^2)$ της brute-force σύγκρισης (Σενάριο 0). Χρησιμοποιήθηκε ο αλγόριθμος **LSH** και συγκεκριμένα η υλοποίηση `MinHashLSH`.

Λειτουργία: Αντί να συγκρίνουμε κάθε μάθημα με όλα τα υπόλοιπα, ο αλγόριθμος υπολογίζει έναν αριθμό από *similarity preserving signatures* για κάθε έγγραφο. Στη συνέχεια, κατανέμει τα μαθήματα με βάση αυτές τις υπογραφές σε κάδους (*buckets*). Μαθήματα που είναι παρόμοια έχουν πολύ μεγάλη πιθανότητα να καταλήξουν στον ίδιο κάδο, περιορίζοντας δραστικά τον αριθμό των συγκρίσεων που πρέπει να γίνουν.

- **Γιατί Jaccard Distance:** Ο αλγόριθμος `MinHash` είναι μαθηματικά σχεδιασμένος να προσεγγίζει τον συντελεστή ομοιότητας **Jaccard**. Η επιλογή αυτή είναι ιδανική για δεδομένα κειμένου (όπως οι περιγραφές μας), καθώς η ομοιότητα Jaccard μετρά την επικάλυψη των συνόλων των λέξεων μεταξύ δύο εγγράφων ($\frac{|A \cap B|}{|A \cup B|}$), δίνοντας έμφαση στην παρουσία κοινών όρων ανεξάρτητα από το μέγεθος του διανύσματος.

- **MinHash vs Bucketed Random Projection (BRP):** Ενώ ο BucketedRandomProjectionLSH χρησιμοποιείται κυρίως για την προσέγγιση της Ευκλείδειας απόστασης σε dense vectors. Τα διανύσματα TF-IDF είναι *sparse*, το MinHash επικεντρώνεται στα μη-μηδενικά στοιχεία (λέξεις που υπάρχουν), προσφέροντας πολύ ταχύτερα αποτελέσματα για την εύρεση παρόμοιων κειμένων.
- **Χρησιμότητα:** Επιτρέπει την εκτέλεση του συστήματος συστάσεων σε πραγματικό χρόνο ή σε περιβάλλοντα με εκατομμύρια εγγραφές, θυσιάζοντας ένα ελάχιστο ποσοστό ακρίβειας για τεράστια κέρδη σε ταχύτητα και πόρους.

Γιατί η Ομοιότητα Συνημιτόνου αποτελεί το Ground Truth; Παρόλο που ο αλγόριθμος MinHash LSH χρησιμοποιεί την απόσταση Jaccard για λόγους ταχύτητας, το Cosine Similarity επιλέχθηκε ως το σημείο αναφοράς για τους εξής λόγους:

1. **Ενσωμάτωση των Βαρών TF-IDF:** Η ομοιότητα Jaccard εξετάζει τα έγγραφα ως σύνολα λέξεων (παρουσία ή απουσία μιας λέξης). Αντίθετα, η Cosine Similarity υπολογίζεται πάνω στα διανύσματα των βαρών TF-IDF. Αυτό σημαίνει ότι λαμβάνει υπόψη τη σχετική σημασία κάθε λέξης. Για παράδειγμα, αν δύο μαθήματα μοιράζονται τη σπάνια λέξη "Kubernetes", η Cosine Similarity θα τους δώσει πολύ υψηλότερο σκορ από ότι αν μοιράζονταν την κοινή λέξη "Introduction", ενώ η Jaccard θα τις αντιμετώπιζε σχεδόν ισότιμα.
2. **Ανθεκτικότητα στο Μέγεθος του Κειμένου (Length Invariance):** Η Cosine Similarity μετράει τη γωνία των διανυσμάτων και όχι το μέγεθός τους. Αυτό την καθιστά ιδανική για περιγραφές μαθημάτων, καθώς ένα μάθημα με πολύ μεγάλη περιγραφή και ένα με σύντομη περιγραφή μπορούν να θεωρηθούν παρόμοια αν "κοιτάζουν" προς την ίδια νοηματική κατεύθυνση, ανεξάρτητα από το πλήθος των λέξεων.
3. **Ακρίβεια έναντι Προσέγγισης:** Το Σενάριο 0 (Exact Cosine) εκτελεί τον πλήρη μαθηματικό υπολογισμό για κάθε πιθανό ζεύγος εγγράφων ($O(N^2)$). Σε αντίθεση με το LSH, το οποίο είναι μια προσεγγιστική μέθοδος που μπορεί να "χάσει" κάποια παρόμοια ζεύγη αν δεν καταλήξουν στον ίδιο κάδο, η exact μέθοδος εγγυάται την εύρεση των απολύτως καλύτερων συστάσεων βάσει του μοντέλου TF-IDF.

5.3.3 Σενάριο 2: Thematic Recommender (LDA + LSH)

Αυτή η προσέγγιση μετατοπίζει την εστίαση από τις μεμονωμένες λέξεις στις «λανθάνουσες» (latent) νοηματικές ενότητες. Αντί να συγκρίνουμε μαθήματα βάσει κοινών όρων, τα συγκρίνουμε βάσει των θεμάτων (*topics*) στα οποία ανήκουν.

Λειτουργία:

- **Μοντελοποίηση LDA:** Χρησιμοποιείται το μοντέλο **LDA (Latent Dirichlet Allocation)**, ένας αλγόριθμος *soft clustering*. Το LDA θεωρεί ότι κάθε μάθημα είναι ένα μείγμα διαφόρων θεμάτων και κάθε θέμα ένα μείγμα διαφόρων λέξεων. Το αποτέλεσμα για κάθε μάθημα είναι ένα διάνυσμα πιθανοτήτων (π.χ. 70% Προγραμματισμός, 20% Βάσεις Δεδομένων, 10% Δίκτυα). Αν και ιδανικό για documents μεγαλύτερης έκτασης όπως research papers και άρθρα, η χρήση του προσφέρει και εδώ κάποια πλεονεκτήματα.
- **Διανύσματα Θεμάτων (Topic Distributions):** Σε αντίθεση με τα διανύσματα TF-IDF που είναι εξαιρετικά αραιά (*sparse*), οι κατανομές θεμάτων του LDA είναι πυκνά διανύσματα (*dense*) μικρής διάστασης (ίσης με τον αριθμό των θεμάτων k). Δύο μαθήματα θεωρούνται παρόμοια αν οι κατανομές των θεμάτων τους «μοιάζουν» στατιστικά.
- **Χρήση BRP LSH:** Λόγω της πυκνής φύσης αυτών των διανυσμάτων, ο υπολογισμός της ομοιότητας μπορεί να γίνει αποδοτικά με τον αλγόριθμο **Bucketed Random Projection (BRP) LSH**. Ο BRP χρησιμοποιεί τυχαίες προβολές για να ομαδοποιήσει τα διανύσματα σε κάδους βάσει της Ευκλείδειας απόστασης ή cosine distance, καθιστώντας την αναζήτηση θεματικά παρόμοιων μαθημάτων εξαιρετικά γρήγορη.
- **Πλεονέκτημα (Εννοιολογική Συνάφεια):** Το κύριο πλεονέκτημα αυτής της μεθόδου είναι η αντιμετώπιση του προβλήματος της συνωνυμίας. Δύο μαθήματα μπορεί να αναφέρονται στο ίδιο αντικείμενο χρησιμοποιώντας εντελώς διαφορετικό λεξιλόγιο (π.χ. το ένα να αναφέρει «ανάπτυξη εφαρμογών» και το άλλο «software engineering»). Το LDA είναι σε θέση να αναγνωρίσει ότι και τα δύο ανήκουν στο ίδιο θέμα, καθώς η στατιστική συνύπαρξη των υπολοίπων λέξεων στο κείμενο (context) προσδιορίζει την κοινή θεματική ενότητα, προσφέροντας συστάσεις που ξεπερνούν τους περιορισμούς των απλών στατιστικών μεθόδων NLP.

5.3.4 Σενάριο 3: Υβριδικό Μοντέλο Συστάσεων (Hybrid Recommender)

Το υβριδικό μοντέλο αποτελεί την πιο ολοκληρωμένη προσέγγιση του συστήματος, καθώς επιχειρεί να συνδυάσει τα πλεονεκτήματα της λεξιλογικής ομοιότητας (TF-IDF) και της θεματικής συνάφειας (LDA), εξαλείφοντας παράλληλα τις αδυναμίες τους.

Λειτουργία και Υλοποίηση:

- **Συνδυασμός Αποτελεσμάτων (Outer Join):** Το σύστημα εκτελεί μια πράξη *Outer Join* μεταξύ των αποτελεσμάτων των δύο προηγούμενων μοντέλων. Με αυτόν τον τρόπο, διασφαλίζεται ότι μια σύσταση θα ληφθεί υπόψη ακόμα και αν εντοπίστηκε μόνο από το ένα εκ των δύο μοντέλων.
- **Σταθμισμένος Μέσος Όρος (Weighted Scoring):** Για κάθε ζεύγος μαθημάτων, υπολογίζεται μια τελική «υβριδική απόσταση» χρησιμοποιώντας συντελεστές βαρύτητας (*weights*). Στην παρούσα υλοποίηση χρησιμοποιήθηκε η αναλογία 60% για το TF-IDF και 40% για το LDA:

$$D_{hybrid} = (w_{tfidf} \cdot Dist_{tfidf}) + (w_{lda} \cdot Dist_{lda}) \quad (3)$$

- **Διαχείριση Ελλιπών Δεδομένων (Penalization):** Σε περιπτώσεις όπου ένα ζεύγος μαθημάτων προτείνεται από το ένα μοντέλο αλλά όχι από το άλλο, το σύστημα εφαρμόζει μια «ποινή» (*penalty*), αποδίδοντας τη μέγιστη δυνατή απόσταση (1.0) στο μοντέλο που δεν εντόπισε τη συνάφεια. Αυτό διασφαλίζει ότι οι πιο ισχυρές συστάσεις είναι αυτές που αναγνωρίζονται και από τις δύο μεθόδους.

Πλεονεκτήματα: Η υβριδική προσέγγιση επιτρέπει στο σύστημα να προτείνει μαθήματα που έχουν τόσο παρόμοιους τίτλους και περιγραφές (μέσω TF-IDF), όσο και παρόμοιο εννοιολογικό υπόβαθρο (μέσω LDA), προσφέροντας μια πιο «φυσική» και ανθρώπινη εμπειρία χρήστη.

5.3.5 Τελική Επιλογή και Κατάταξη

Ανεξάρτητα από τη μέθοδο υπολογισμού που ακολουθείται (Exact, LSH, LDA ή Hybrid), το τελευταίο στάδιο του αγωγού επεξεργασίας είναι κοινό για όλα τα σενάρια. Σκοπός είναι ο περιορισμός του όγκου των δεδομένων και η διατήρηση μόνο των πιο συναφών προτάσεων για τον τελικό χρήστη.

Διαδικασία:

- **Χρήση Window Functions:** Για την αποτελεσματική διαχείριση των δεδομένων σε περιβάλλον Spark, χρησιμοποιούνται οι συναρτήσεις παραθύρου (*Window Functions*). Αυτό επιτρέπει στο σύστημα να ομαδοποιεί τις συστάσεις ανά μάθημα-πηγή (*id_a*) και να τις ταξινομεί εσωτερικά.
- **Φιλτράρισμα Top-5:** Εφαρμόζεται η συνάρτηση `row_number()` για την απόδοση μιας σειράς κατάταξης σε κάθε πρόταση βάσει του *score* ομοιότητας (ή της απόστασης). Στη συνέχεια, το σύστημα φιλτράρει και κρατά μόνο τις 5 επικρατέστερες συστάσεις ($rank \leq 5$).

```
window_spec = Window.partitionBy("id_a").orderBy(fun.col("distance").asc())

lsh_knn_results = flattened_lsh \
    .withColumn("rank", fun.row_number().over(window_spec)) \
    .filter(fun.col("rank") <= 5)
```

5.4 Θεματική Συσταδοποίηση (Topic-based Clustering)

Αντί για την παραδοσιακή συσταδοποίηση (όπως ο K-Means), το σύστημα χρησιμοποιεί το μοντέλο LDA (Latent Dirichlet Allocation) για την οργάνωση των μαθημάτων. Η προσέγγιση αυτή είναι πιο εξελιγμένη καθώς αντιμετωπίζει τα μαθήματα ως μείγματα θεμάτων και όχι ως στατικά αντικείμενα που ανήκουν αυστηρά σε μία ομάδα.

Λειτουργία και Υλοποίηση:

- **Soft vs Hard Clustering:** Σε αντίθεση με παραδοσιακούς αλγόριθμους όπως ο K-Means, οι οποίοι εκτελούν «σκληρή» συσταδοποίηση (*hard clustering*) αναθέτοντας κάθε μάθημα αποκλειστικά σε μία ομάδα, το LDA υιοθετεί μια πιθανοτική προσέγγιση. Θεωρεί ότι κάθε μάθημα αποτελεί ένα **μείγμα (mixture)** διαφορετικών θεμάτων. Για παράδειγμα, ένα μάθημα "Machine Learning in Finance" μπορεί να αναλυθεί από το LDA ως 60% "Data Science" και 40% "Finance", αντί να εξαναγκαστεί σε μία μόνο από τις δύο κατηγορίες.
- **Εξαγωγή Κύριας Συστάδας (Maximum Likelihood):** Παρόλο που το μοντέλο παράγει μια κατανομή πιθανοτήτων, για τις ανάγκες της οργάνωσης της βάσης

δεδομένων, το σύστημα απομονώνει το θέμα με την υψηλότερη πιθανότητα (*argmax*). Αυτό το επικρατέστερο θέμα χρησιμοποιείται ως η «κύρια συστάδα» (*cluster_label*), επιτρέποντας την κατηγοριοποίηση των μαθημάτων χωρίς να χάνεται η εσωτερική πληροφορία της θεματικής συνάφειας που χρησιμοποιείται στο Σενάριο 2.

- **Ερμηνευσιμότητα και Εξαγωγή Θεματικών Τίτλων:** Ένα καθοριστικό πλεονέκτημα του LDA έναντι του K-Means είναι η δυνατότητα απόδοσης νοήματος στις συστάδες. Ενώ ο K-Means παράγει ανώνυμες ομάδες (π.χ. «Cluster 1», «Cluster 2»), καθιστώντας δύσκολη την κατανόηση του περιεχομένου τους χωρίς περαιτέρω ανάλυση, το LDA παρέχει για κάθε θέμα μια λίστα με τις επικρατέστερες λέξεις (*top words*).

Για παράδειγμα, αν ένα θέμα περιλαμβάνει τις λέξεις "algorithm", "data", "structure", ο προγραμματιστής ή ο τελικός χρήστης μπορεί άμεσα να αντιληφθεί ότι η συστάδα αφορά την Πληροφορική. Αυτή η δυνατότητα επέτρεψε στο σύστημα να "τιτλοφορεί" εμμέσως τις κατηγορίες, προσφέροντας μια πολύ πιο διαφανή και φιλική προς τον χρήστη εμπειρία.

- **Υλοποίηση στο Spark:** Η διαδικασία υλοποιείται μέσω του `pyspark.ml.clustering.LDA`. Τα αποτελέσματα μετασχηματίζονται ώστε να εξαχθεί ο δείκτης (*index*) του επικρατέστερου θέματος.

Η Σημασία του Clustering στις Συστάσεις: Η πληροφορία της συστάδας δεν χρησιμοποιείται μόνο για προβολή, αλλά ενισχύει την ακρίβεια των συστάσεων με τον εξής τρόπο:

1. **Επανατροφοδότηση Θεματικών Τίτλων στον Recommender (Semantic Boosting):** Μια από τις πιο σημαντικές καινοτομίες στην αρχιτεκτονική του συστήματος είναι η χρήση της πληροφορίας των συστάδων ως "τροφή" για τον αλγόριθμο ομοιότητας. Η ετικέτα της συστάδας (*cluster_label*) που προέκυψε από το μοντέλο LDA δεν χρησιμοποιήθηκε απλώς ως μεταδεδομένο, αλλά επανεντάχθηκε στο κείμενο προς επεξεργασία.
2. **Feature Boosting:** Στο στάδιο της προετοιμασίας του κειμένου (Preprocessing), η ετικέτα της συστάδας (*cluster_label*) προστίθεται επαναλαμβανόμενες φορές στο πεδίο *text_content*. Η επανάληψη αυτή είναι σκόπιμη, καθώς αυξάνει τεχνητά

τη συχνότητα εμφάνισης του όρου (*Term Frequency - TF*), δίνοντάς του μεγαλύτερο βάρος στο τελικό διάνυσμα TF-IDF.

Αποτέλεσμα στις Συστάσεις: Με αυτή την τεχνική, επιτυγχάνεται η «σημασιολογική ενίσχυση» των συστάσεων. Ακόμα και αν δύο μαθήματα έχουν περιγραφές με εντελώς διαφορετικό λεξιλόγιο, η κοινή τους υπαγωγή στο ίδιο θεματικό cluster λειτουργεί ως "συνδετικός κρίκος". Αυτό αναγκάζει τα διανύσματά τους να συγκλίνουν στον πολυδιάστατο χώρο, διασφαλίζοντας ότι ο Recommender θα προτείνει μαθήματα που είναι θεματικά συγγενή, μειώνοντας τις πιθανότητες εμφάνισης άσχετων προτάσεων λόγω τυχαίας σύμπτωσης λέξεων.

5.5 Προσωποποιημένες Προτάσεις Χρηστών με χρήση ALS

Τέλος, ένα ακόμη Spark Job που υλοποιήθηκε είναι εκείνο της πρότασης νέων μαθημάτων σε χρήστες, βάσει των μαθημάτων που έχουν 'σημαδέψει' ως αγαπημένα (Favorites). Οι προτάσεις γίνονται μέσω του ALS (Alternating Least Squares), μιας μεθόδου που ανήκει στην οικογένεια των Collaborative Filtering αλγορίθμων.

5.5.1 Γενική Περιγραφή ALS

Η μέθοδος ALS στην ουσία βασίζεται στην παραγοντοποίηση ενός μεγάλου και αραιού μητρώου R (user-item interactions) στο γινόμενο 2 μητρώων:

- U : 'Latent' Προτιμήσεις Χρηστών
- V^T : 'Latent' Χαρακτηριστικά Items

Αυτό που την διαφοροποιεί από άλλες μεθόδους είναι η εναλλαγή που κάνει ως προς την επίλυση μέσω Ελάχιστων Τετραγώνων (εξού και Alternating Least Squares). Σε κάθε επαναλήψη 'φιζάρουμε' ένα από τα 2 μητρώα (U και V) και λύνουμε ως προς το άλλο, φροντίζοντας να μην λύσουμε ποτέ προς το ίδιο μητρώο σε 2 συνεχόμενες επαναλήψεις. Μετά από κάποιον αριθμό επαναλήψεων και όταν η συνάρτηση κόστους επιστρέφει ικανοποιητικά αποτελέσματα, έχουμε δύο μητρώα U' και V' , όπου πολλαπλασιάζοντάς τα προκύπτει το νέο μητρώο R' το οποίο είναι πλέον πυκνό. Για κάθε χρήστη και κάθε item (κελί u,i) παράγεται ένα confidence score που υποδεικνύει το πόσο κοντά είναι το αντικείμενο αυτό στις προτιμήσεις του χρήστη.

5.5.2 Εφαρμογή και υλοποίηση στην πρόταση μαθημάτων

Στην δική μας περίπτωση, θέλουμε να προτείνουμε στον κάθε χρήστη ένα σύνολο μαθημάτων που θα μπορούσε να επιλέξει, βάσει των αγαπημένων του. Αξιοσημείωτο είναι το γεγονός πως τα μαθήματα δεν έχουν ratings (π.χ. 2/5, όπως οι ταινίες) αλλά μια binary συμβολοποίηση, εκείνη που είτε σε κάποιον χρήστη αρέσει (1) ή όχι (0) ένα μάθημα. Τα βήματα της υλοποίησης μας είναι τα εξής:

1. Παίρνουμε τα δεδομένα από την βάση μας και τα τοποθετούμε σε ένα DataFrame.
2. Αφού σημαδεύουμε κάθε course με 0 ή 1 για κάθε χρήστη, μετατρέπουμε τα IDs των χρηστών και των μαθημάτων από Strings σε Integers, για να εφαρμόσουμε ALS.
3. Εφαρμόζουμε ALS στο μοντέλο μας και κρατάμε τις 10 καλύτερες προτάσεις για κάθε χρήστη.
4. Μαζεύουμε τα IDs μαθημάτων του κάθε χρήστη και κάνουμε την αντίστροφη διαδικασία της προηγούμενης μετατροπής (Integer IDs $\rightarrow$ String IDs).
5. Κάνουμε export τα αποτελέσματα σε έναν νέο πίνακα στην βάση δεδομένων μας, με όνομα `user_recommendations`.

Ο admin μπορεί να εκτελέσει το spark job στο admin panel και οι προτάσεις εμφανίζονται μαζί με τα αγαπημένα μαθήματα στον κάθε χρήστη.

5.6 Τεχνικές Προκλήσεις και Επίλυση Προβλημάτων

Κατά την ανάπτυξη και την υλοποίηση του συστήματος προέκυψαν σημαντικά τεχνικά εμπόδια που αφορούσαν το περιβάλλον εκτέλεσης, τη διαχείριση των δεδομένων και τη συμβατότητα των βιβλιοθηκών του Spark.

5.6.1 Μετάβαση από Windows σε WSL

Αρχικά, η εκτέλεση του Spark σε περιβάλλον Windows παρουσίασε κρίσιμα σφάλματα επικοινωνίας μεταξύ του Driver και των Executors, τα sockets εκρηγνύονταν με λίγα λόγια (SocketTimeout και WinError 10038).

- **Πρόβλημα:** Η έκδοση της Python από το Windows Store και οι περιορισμοί του δικτύου των Windows εμπόδιζαν τη σωστή λειτουργία των *worker processes* του Spark.
- **Λύση:** Το σύστημα μεταφέρθηκε σε περιβάλλον **WSL (Ubuntu)**, το οποίο προσφέρει πλήρη συμβατότητα με τα POSIX συστήματα που απαιτεί το Spark. Έγινε χρήση εικονικού περιβάλλοντος (*venv*) και ορίστηκαν ρητά οι μεταβλητές περιβάλλοντος `PYSPARK_PYTHON` και `PYSPARK_DRIVER_PYTHON` για τη διασφάλιση της ομοιομορφίας των εκδόσεων Python.

5.6.2 Διαχείριση Αναγνωριστικών MongoDB (`_id`)

Η ενσωμάτωση του Spark με τη MongoDB (μέσω του MongoDB Spark Connector) παρουσίασε ιδιαιτερότητες στη διαχείριση του πεδίου `_id`.

- **Πρόβλημα:** Το Spark αναπαριστά το `ObjectId` της MongoDB ως σύνθετο `StructType` (`oid: string`), γεγονός που καθιστούσε αδύνατη τη χρήση του ως κλειδί για συσχετίσεις (*joins*) ή την επανεγγραφή του στη βάση.
- **Λύση:** Κατά την ανάγνωση των δεδομένων, έγινε χρήση εναλλακτικών μοναδικών αναγνωριστικών (όπως το `externalId` το οποίο αναφέρεται απλά ως `course_id`), διασφαλίζοντας ότι οι συστάσεις αποθηκεύονται με τρόπο που επιτρέπει την εύκολη ανάκτησή τους από το Backend.

5.6.3 Ασυμβατότητα Βιβλιοθηκών `pyspark.ml` και `pyspark.mllib`

Ένα από τα πιο σύνθετα προβλήματα προέκυψε κατά την υλοποίηση του Σεναρίου 0 (Exact Similarity).

- **Πρόβλημα:** Η σύγχρονη βιβλιοθήκη `pyspark.ml` (`DataFrames`) παράγει διανύσματα τύπου `SparseVector` που δεν είναι άμεσα συμβατά με τη χαμηλού επιπέδου βιβλιοθήκη `pyspark.mllib`, η οποία απαιτείται για τη δημιουργία του `IndexedRowMatrix` και την εκτέλεση κατανεμημένου πολλαπλασιασμού πινάκων.
- **Λύση:** Υλοποιήθηκε ένας μηχανισμός μετατροπής των διανυσμάτων μέσα στο RDD pipeline, χρησιμοποιώντας τη μέθοδο `Vectors.fromML()`. Αυτό επέτρεψε τη ροή

των δεδομένων από το NLP pipeline (DataFrames) στον κινητήρα γραμμικής άλγεβρας (RDD Matrices).

```
# Η λύση στο πρόβλημα συμβατότητας των διανυσμάτων
from pyspark.mllib.linalg import Vectors as OldVectors
rdd_vectors = df.rdd.map(lambda row: IndexedRow(row.id,
                                                    OldVectors.fromML(row.features)))
```

5.7 Σχεδιαστικές Αποφάσεις και Μελλοντική Επεκτασιμότητα

- **Αρχικός Σχεδιασμός (Static Batch Processing):** Αρχικά έγινε η υπόθεση ότι η εφαρμογή θα λειτουργούσε στατικά. Σύμφωνα με αυτό το σενάριο, θα γινόταν *harvest* ένα προκαθορισμένο σύνολο δεδομένων από APIs, θα εκτελούνταν εφάπαξ τα Spark Jobs (`clusterer.py` και `recommender.py`) και η εφαρμογή θα κατέληγε σε ένα παραδοσιακό Web App που ανακτά έτοιμα αποτελέσματα από τη MongoDB.
- **Προβλήματα Κλιμακωσιμότητας (Scalability Bottlenecks):** Μετά από ανάλυση των πραγματικών συνθηκών, καταλήξαμε ότι ένα τέτοιο σύστημα θα δεχόταν νέο περιεχόμενο με εξαιρετικά υψηλό ρυθμό. Η ανάγκη για επανεκτέλεση ολόκληρου του Spark Job (Batch) κάθε φορά που προστίθεται ένα μάθημα είναι υπολογιστικά ασύμφορη, ειδικά σε μη αποκεντρωμένα συστήματα. Σε περιβάλλοντα παραγωγής, η αναμονή για την ολοκλήρωση ενός Batch Job για την παραγωγή μιας σύστασης θα καθιστούσε το σύστημα μη αποκρίσιμο.
- **Μετάβαση σε Αρχιτεκτονική Kappa και χρήση Kafka:** Για την αντιμετώπιση αυτού του προβλήματος, η ιδανική εξέλιξη του συστήματος βασίζεται στην **Αρχιτεκτονική Kappa (Kappa Architecture)**. Σε αυτή την προσέγγιση:
 - Το **Apache Kafka** λειτουργεί ως ο κεντρικός δίαυλος (*Unified Log*), όπου κάθε νέο μάθημα εισέρχεται ως ένα συμβάν (event) σε πραγματικό χρόνο.
 - Το Spark δεν επεξεργάζεται πλέον όλη τη βάση από την αρχή, αλλά χρησιμοποιεί το **Spark Structured Streaming** για να επεξεργάζεται σταδιακά (*incrementally*) τις νέες εγγραφές.
 - Αντί για επανυπολογισμό ολόκληρου του πίνακα ομοιότητας, το σύστημα εκτελεί *Online Scoring* ή χρησιμοποιεί προσεγγιστικούς αλγόριθμους (όπως το LSH που

υλοποιήθηκε) για να εντάξει το νέο μάθημα στα υπάρχοντα clusters αχαριαία.

5.8 Τελική Προσέγγιση: Triggered Batch Architecture (Admin-Driven)

Στην τελική υλοποίηση, το σύστημα ακολουθεί μια αρχιτεκτονική **Triggered Batch**. Αντί οι διαδικασίες συλλογής και επεξεργασίας να εκτελούνται αυτόματα ή αποκλειστικά σε προκαθορισμένα χρονικά διαστήματα, ο έλεγχος δίνεται στον διαχειριστή (*Admin*) μέσω ενός ειδικού περιβάλλοντος διεπαφής (*Dashboard*).

Λογική και Πλεονεκτήματα:

- **On-demand Harvesting:** Ο διαχειριστής έχει τη δυνατότητα να ενεργοποιεί τη συλλογή δεδομένων από τους `API Connectors` μόνο όταν είναι απαραίτητο (π.χ. όταν υπάρχουν νέες πηγές). Αυτό αποτρέπει την άσκοπη κατανάλωση πόρων και τον περιορισμό από τα *rate limits* των εξωτερικών συστημάτων.
- **Ελεγχόμενη Εκτέλεση Spark Jobs:** Όπως φαίνεται στο UI της εφαρμογής, η εκτέλεση των αλγορίθμων (Exact TF-IDF, Fast LSH, LDA, Hybrid) γίνεται κατ' επιλογή. Αυτό είναι ιδιαίτερα κρίσιμο σε περιβάλλοντα Big Data, καθώς επιτρέπει στον διαχειριστή να εκτελεί το βαρύ υπολογιστικό κομμάτι σε ώρες χαμηλού φόρτου συστήματος.
- **Διαχωρισμός Επιπέδων (Decoupling):** Η αρχιτεκτονική αυτή διαχωρίζει πλήρως το επίπεδο συλλογής (*Data Ingestion*) από το επίπεδο επεξεργασίας (*Spark Engine*). Ο διαχειριστής μπορεί να ολοκληρώσει το harvesting από πολλαπλές πηγές (π.χ. `api-1`, `api-2`) και στη συνέχεια να αποφασίσει ποιο συγκεκριμένο μοντέλο σύστασης θα «εκπαιδεύσει» πάνω στα νέα δεδομένα.

Σύνδεση με Μελλοντική Επεκτασιμότητα: Η συγκεκριμένη «χειροκίνητη» προσέγγιση (*Human-in-the-loop*) αποτελεί το απαραίτητο ενδιάμεσο στάδιο προς τον πλήρη αυτοματισμό. Η δομή του κώδικα είναι σχεδιασμένη ώστε τα Spark Jobs να μπορούν να μετατραπούν σε προγραμματισμένες εργασίες (*Scheduled Tasks*) ή σε *Streaming Consumers* (μέσω Apache Kafka) με ελάχιστες τροποποιήσεις, καθώς η λογική της επεξεργασίας παραμένει modular και ανεξάρτητη από τον τρόπο ενεργοποίησης.

6 Συμπεράσματα και Συνολική Αποτίμηση

Η παρούσα εργασία αποτέλεσε μια ολοκληρωμένη προσπάθεια σχεδιασμού και υλοποίησης ενός end-to-end συστήματος οριζόντιου repository/aggregator μαθημάτων. Στόχος ήταν η δημιουργία μιας υποδομής ικανής να συλλέγει, να αποθηκεύει και να αξιολογεί πειραματικά δεδομένα μεγάλης κλίμακας[cite: 3, 4], όπως αυτά των περιγραφών μαθημάτων και μεταδεδομένων.

6.1 Η Σύγκλιση των Τεχνολογιών

Το τελικό αποτέλεσμα συνδυάζει διαφορετικά επίπεδα αρχιτεκτονικής, δημιουργώντας ένα πλήρες οικοσύστημα επεξεργασίας:

- **Data Orchestration (Back-end):** Το Back-end λειτούργησε ως η «γέφυρα» του συστήματος, διαχειριζόμενο τη ροή πληροφοριών από τα εξωτερικά APIs προς τη MongoDB και συντονίζοντας την επικοινωνία με τον κινητήρα του Spark. Η αρχιτεκτονική του επέτρεψε την ασφαλή αποθήκευση και τη γρήγορη ανάκτηση των συστάσεων.
- **Machine Learning Engine (Spark):** Ο κινητήρας επεξεργασίας ανέλαβε το βαρύ υπολογιστικό έργο μοιράζοντας το στους worker nodes και παραλληλοποιώντας το. Μέσα από τα υβριδικά μοντέλα (TF-IDF και LDA), κατάφερε να μετατρέψει τον ακατέργαστο όγκο περιγραφών σε δομημένη γνώση και συναφείς προτάσεις, αποδεικνύοντας την αξία του Apache Spark σε προβλήματα μεγάλης κλίμακας.
- **Modern User Interface (React Frontend):** Η διεπαφή χρήστη αναπτύχθηκε με τη χρήση της βιβλιοθήκης React, προσφέροντας μια σύγχρονη και responsive εμπειρία. Το Admin Dashboard επιτρέπει στον χρήστη να ενορχηστρώνει τη ροή των δεδομένων από το harvesting έως το clustering καθιστώντας τις σύνθετες καταναεμένες διεργασίες προσβάσιμες μέσω μιας απλής διεπαφής.

6.2 Τελικό Συμπέρασμα

Το project παραδίδεται ως μια επεκτάσιμη πλατφόρμα, έτοιμη να ενσωματώσει νέες πηγές δεδομένων και να μεταβεί σε πραγματικό χρόνο επεξεργασία (*Streaming*), επιβεβαιώνοντας

ότι ο συνδυασμός σύγχρονων εργαλείων *Data Engineering* και *Web Development* μπορεί να δώσει λύσεις σε απαιτητικά προβλήματα της σύγχρονης αγοράς.

7 Github Repository

Όλος το source code της υλοποίησης μπορεί να βρεθεί στο GitHub αποθετήριο μας:

https://github.com/nderekk/Spark_Aggregator